FairWindSK

Owner’s Manual

Your onboard display shell for Signal K navigation

v0.9.84 | `f19e3a2` | 2026-05-16

Complete operational guide for sailors

Edition 2.1

OpenFairWind Project

University of Naples ’Parthenope’ · DYNAMO

Built 2026-05-16

FairWindSK supports onboard situational awareness. It does not replace lookout, seamanship, paper charts, or certified safety procedures.

FairWindSK Owner's Manual — Edition 2.1

© 2026 OpenFairWind Project, University of Naples 'Parthenope'

Creative Commons Attribution 4.0 International

https://github.com/OpenFairWind/FairWindSK

Software:	0.9.84	**Commit:**	`f19e3a2`
Branch:	`main`	**Count:**	984
Released:	2026-05-16	**Built:**	2026-05-16

Safety notice: This manual describes a software aid to navigation. The master of the vessel bears full responsibility for safe conduct of the passage at all times.

0 Contents

I Getting Started **11**

1 Introduction **13**
1.1 What FairWindSK Does . . . 13
1.1.1 Supported Platforms . . . 13
1.2 What FairWindSK Is NOT . . . 14
1.3 What You Need Before Starting . . . 14
1.4 Device Guidance . . . 14
1.4.1 Tablets and helm displays . . . 14
1.4.2 Phones . . . 14
1.4.3 Desktop systems . . . 14

2 Before You Leave the Dock **17**
2.1 First-Time Setup . . . 17
2.2 Pre-Departure Checklist . . . 17

II The Interface **19**

3 Screen Layout **21**
3.1 The Three Zones . . . 21
3.2 The Application Area . . . 21
3.3 Drawers . . . 22
3.4 Single-Window Model . . . 22

4 The Top Bar **23**
4.1 Fixed Top Bar Elements . . . 23
4.2 Data Widgets — Complete Reference . . . 23
4.3 Reading Navigation Values Correctly . . . 24
4.3.1 COG and SOG . . . 24
4.3.2 Heading . . . 25
4.3.3 Route and waypoint values . . . 25
4.4 Configuring the Top Bar . . . 25

5 The Bottom Bar **27**
5.1 Bottom Bar Regions . . . 27
5.2 Main Icon Reference . . . 27

5.3 Operational Overlay Bars 28

6 The Launcher and Application Management 29
6.1 Opening an Application 29
6.2 Switching Between Open Applications 29
6.3 Adding a Custom Application 30
6.4 Recovering a Stuck or Blank Application 30

III Operational Features 31

7 Signal K and Data Health 33
7.1 Individual Value States 33
7.2 Connection Health States 33
7.3 When to Act Immediately 34
7.4 Automatic Recovery Cycle 34

8 Person Over Board (POB) 37
8.1 Activating POB 37
8.2 POB Bar Controls and Readouts 38
8.3 SAR Search Patterns 38
8.3.1 Spiral Search 38
8.3.2 Parallel-Track Search 39
8.4 Cancelling a POB 39

9 Autopilot Controls 41
9.1 Autopilot Bar Controls 41
9.2 Rudder Indicator 42

10 Anchor Watch 43
10.1 Anchor Bar Controls and Readouts 43
10.2 Setting Anchor Watch — Step by Step 44

11 Alarms 45
11.1 Emergency Alarm Buttons 45
11.2 Alarm Priority and Response 46
11.3 Active Alarm List 46

IV My Data 47

12 My Data — Overview 49
12.1 My Data Tab Overview 49
12.2 The Common Toolbar 50

13 Waypoints 53
13.1 Waypoints Table Columns 53

13.2 Searching Waypoints . . . 54
13.3 Creating a Waypoint . . . 54
13.3.1 Editor Fields . . . 54
13.4 Editing a Waypoint . . . 55
13.5 Deleting a Waypoint . . . 55
13.6 Opening a Waypoint — the Detail View . . . 55
13.7 Navigating to a Waypoint . . . 56
13.8 Importing Waypoints . . . 56
13.9 Exporting Waypoints . . . 56

14 Routes **57**
14.1 Routes Table Columns . . . 57
14.2 Opening a Route — the Detail View . . . 57
14.3 Creating and Editing Routes . . . 58
14.3.1 Creating a Route . . . 58
14.3.2 Activating a Route . . . 58
14.4 Importing and Exporting Routes . . . 59

15 Regions **61**
15.1 Regions Table Columns . . . 61
15.2 Creating a Region . . . 61

16 Notes **63**
16.1 Notes Table Columns . . . 63
16.2 Creating a Note . . . 64
16.3 Opening a Note . . . 64
16.3.1 Notes Without an Attachment . . . 64
16.3.2 Notes With an Attachment (href set) . . . 64
16.4 Importing Notes . . . 65

17 Charts **67**
17.1 Charts Table Columns . . . 67
17.2 Adding a Chart Source . . . 68
17.3 Importing a Chart Source . . . 68
17.4 Exporting the Chart Catalogue . . . 68

18 Tracks — History **71**
18.1 Track Table Columns . . . 71
18.2 Live Track Recording . . . 71
18.3 Viewing a Track Point . . . 72
18.3.1 Editing a Track Point . . . 72
18.4 Adding a Track Point Manually . . . 72
18.5 Deleting a Track Point . . . 72
18.6 Exporting the Track . . . 72
18.7 Importing a Track . . . 73

18.8 Uses of the Track Tab . . . 73

19 Files **75**
19.1 Interface Layout . . . 75
19.2 Navigation . . . 75
19.2.1 Directory Navigation . . . 75
19.2.2 Column Sorting . . . 76
19.3 Searching Files . . . 76
19.3.1 Basic Search . . . 76
19.3.2 Search Options . . . 76
19.4 File Actions . . . 77
19.4.1 Opening Files . . . 77
19.4.2 Renaming Files and Folders . . . 78
19.4.3 Creating a New Folder . . . 78
19.4.4 Copy, Cut, and Paste . . . 78
19.4.5 Deleting Files and Folders . . . 78
19.5 File List Columns . . . 78
19.6 Practical Uses of the Files Tab . . . 79

V Settings **81**

20 Settings — Overview **83**

21 Settings — Main **85**
21.1 Tab 0 — Main . . . 85
21.2 Window Mode . . . 85
21.3 Language . . . 86
21.3.1 How localisation works . . . 86
21.3.2 Adding or improving a language . . . 86
21.4 UI Scale Preset . . . 87
21.5 Comfort View Preset and Automatic Switching . . . 87
21.6 Launcher Grid . . . 87
21.7 Coordinate Format . . . 87
21.8 Virtual Keyboard . . . 88

22 Settings — Top Bar and Bottom Bar Editors **89**
22.1 Tab 1 — Top Bar . . . 89
22.2 Tab 2 — Bottom Bar . . . 89
22.3 Live Preview Strip . . . 89
22.4 Editing Toolbar Buttons . . . 89
22.5 Display Toggles . . . 90
22.6 Widget Palette . . . 90

23 Settings — Widgets **91**
23.1 Tab 3 — Widgets . . . 91

23.2 Widget Definition Fields . . . 91

24 Settings — Comfort **93**
24.1 Tab 4 — Comfort . . . 93
24.2 The Six Comfort Presets . . . 93
24.3 Action Buttons . . . 94
24.4 Colour Override Properties . . . 94

25 Settings — Connection **97**
25.1 Tab 5 — Connection . . . 97
25.2 Server URL Field . . . 97
25.3 Status Line . . . 97
25.4 Action Buttons . . . 97
25.4.1 Connect and Pause . . . 98
25.4.2 Request Token . . . 98
25.4.3 Cancel . . . 98
25.4.4 Remove Token . . . 98
25.5 Console Log . . . 98
25.6 mDNS Automatic Discovery . . . 98

26 Settings — Signal K Paths **99**
26.1 Tab 6 — Signal K Paths . . . 99
26.2 Key Path Mappings . . . 99

27 Settings — Units **101**
27.1 Tab 7 — Units . . . 101
27.2 How Unit Preferences Work . . . 101

28 Settings — Applications **103**
28.1 Tab 8 — Applications . . . 103
28.2 Page Tree Actions . . . 103
28.3 Palette Actions . . . 103
28.4 App Details Fields . . . 104

29 Settings — System **105**
29.1 Tab 9 — System . . . 105
29.2 Performance Pane . . . 105
29.2.1 Memory Readouts . . . 105
29.2.2 CPU Load . . . 105
29.2.3 Network Addresses . . . 106
29.2.4 Raspberry Pi Diagnostics . . . 106
29.3 Diagnostics and Logging Pane . . . 106
29.3.1 Log Level . . . 106
29.3.2 Persistent Logs . . . 107
29.3.3 Interaction History . . . 107
29.3.4 View Logs and View Reports Buttons . . . 107

29.4 Configuration Management Buttons 107
29.5 System Control Buttons 108

VI Reference 109

30 Comfort Presets and Visibility 111
30.1 Practical Guidance 111

31 Display, Touch, and Rough-Conditions Operation 113
31.1 Display Readability 113
31.2 Touch Use in Rough Conditions 113

32 Recommended Operator Workflow 115
32.1 Before Departure 115
32.2 During Passage 115
32.3 After a Reconnect or Server Restart 115
32.4 Task Switching 116

33 Signal K Server on Docker 117
33.1 Official Docker Image 117
33.2 Quick Start with Demo NMEA Data 117
33.3 Production Deployment 117
33.4 Updating Signal K 117

34 Troubleshooting 119
34.1 I See Missing Values 119
34.2 I See Stale Values 119
34.3 A Web App Is Blank or Frozen 119
34.4 The Shell Is Disconnected 120
34.5 No Applications Appear on the Launcher 120
34.6 Autopilot or Anchor Icon Is Greyed Out 120
34.7 FairWindSK Running Slowly on Raspberry Pi 120

35 Daily Operating Checklist 121

36 Quick Reference Card 123

37 Safety Notes 125

A Installation 127
A.1 Prerequisites 127
A.1.1 Common prerequisites for all desktop platforms 127
A.1.2 Required Qt 6 modules — desktop 127
A.1.3 Required Qt 6 modules — Android and iOS 128
A.2 Generic Desktop Build (all platforms) 128
A.3 macOS Installation 129

A.3.1 Requirements . . . 129
A.3.2 Install with Homebrew . . . 129
A.3.3 Run from the build tree . . . 130
A.3.4 Install to the system . . . 130
A.3.5 Redistribute (deploy Qt runtime) . . . 130
A.3.6 Post-install verification . . . 130
A.4 Linux Desktop Installation . . . 130
A.4.1 Requirements (Debian / Ubuntu) . . . 130
A.4.2 Build and run . . . 130
A.4.3 System install . . . 131
A.4.4 Packaging . . . 131
A.5 Raspberry Pi OS — Navigation Station . . . 131
A.5.1 Requirements . . . 131
A.5.2 Install build dependencies . . . 131
A.5.3 Build and install . . . 132
A.5.4 OpenPlotter integration . . . 132
A.6 Raspberry Pi OS — Kiosk Helm Display . . . 132
A.6.1 Overview . . . 132
A.6.2 Step 1 — Build and install . . . 133
A.6.3 Step 2 — Configure full-screen kiosk mode . . . 133
A.6.4 Step 3 — Install the startup helper . . . 133
A.6.5 Step 4 — Hide the desktop panel (labwc / Bookworm) . . . 133
A.6.6 Step 5 — Enable virtual keyboard (touchscreen only) . . . 134
A.6.7 Step 6 — Verify the kiosk boot . . . 134
A.7 Windows Installation . . . 134
A.7.1 Requirements . . . 134
A.7.2 Build and install . . . 135
A.7.3 Deploy the Qt runtime . . . 135
A.7.4 Run . . . 135
A.8 Android Installation . . . 135
A.8.1 Requirements . . . 135
A.8.2 Step-by-step build guide . . . 135
A.8.3 Command-line configure (advanced) . . . 136
A.8.4 Android feature differences . . . 136
A.9 iOS and iPadOS Installation . . . 137
A.9.1 Requirements . . . 137
A.9.2 Step-by-step build guide . . . 137
A.9.3 Command-line configure (advanced) . . . 137
A.9.4 iOS feature differences . . . 138
A.10 Signal K Server on Docker . . . 138
A.10.1 Docker image . . . 138
A.10.2 Quick start with demo NMEA data . . . 138
A.10.3 Production deployment . . . 139
A.10.4 Updating Signal K . . . 139

A.10.5 Directory structure inside the container 139
A.11 First-Run Configuration 139
A.11.1 Configuration file locations 139
A.11.2 Connect to Signal K 140
A.11.3 Authenticate (optional) 140
A.11.4 Essential first-time settings 140
A.12 Key Configuration Fields 141
A.12.1 Connection 141
A.12.2 Window mode 141
A.12.3 Units 141
A.12.4 Signal K path overrides 142
A.12.5 Diagnostics 142
A.13 Troubleshooting Installation Problems 142
A.13.1 FairWindSK does not start after installation 142
A.13.2 No applications appear on the launcher 142
A.13.3 Build fails with missing Qt module 143
A.13.4 FairWindSK does not autostart on Raspberry Pi 143

Part I

Getting Started

1 Introduction

FairWindSK is a touch-first marine display shell built around Signal K. It brings together live boat data, a suite of onboard web applications, and native vessel-operation tools inside a single persistent frame designed for the helm.

> FairWindSK is part of the DYNAMO research programme at the University of Naples 'Parthenope', now supported by the DataX4Sea project funded by NEC Laboratories America. The goal is coastal environmental data crowdsourcing through a fleet of leisure boats. All results are open-source and open-data.

1.1 What FairWindSK Does

FairWindSK is a display *shell*, not a data server. It wraps a running Signal K server, discovers the web applications installed on it, and presents them in a stable, persistent frame alongside native operational controls.

Think of FairWindSK as the frame around your boat's apps and live navigation data. It helps you keep key information visible while moving between tasks.

1.1.1 Supported Platforms

Platform	Typical use	Notes
Raspberry Pi OS	Dedicated helm display, kiosk autostart	Full desktop feature set; kiosk helpers in `extras/`
Linux	Navigation station laptop or PC	Full desktop feature set
macOS	Navigation station MacBook	Full desktop feature set
Windows	Navigation station PC	Full desktop feature set
Android	Helm tablet, secondary display	Qt WebView mobile path

iOS / iPadOS	iPad at helm or roving display	Qt WebView mobile path

1.2 What FairWindSK Is NOT

FairWindSK depends entirely on a running Signal K server on your onboard network. Without Signal K it has no data to display, no apps to launch, and no instruments to read. It does not itself process NMEA sentences, connect directly to instruments, or store charts.

> FairWindSK is an aid to navigation. It does not replace seamanship, lookout, paper charts, or certified safety procedures. The master of the vessel bears full responsibility for safe conduct of the passage at all times.

1.3 What You Need Before Starting

- A running Signal K server reachable on the boat's Wi-Fi network. The server must expose the standard API at `<server>/signalk` and the app catalogue at `<server>/signalk/v1/apps/list`.
- Your instruments (GPS, wind, depth, AIS, autopilot, engine monitor) feeding into Signal K via NMEA 0183, NMEA 2000, or SeaTalk.
- FairWindSK installed on a display device.

1.4 Device Guidance

1.4.1 Tablets and helm displays

Tablets and larger marine displays are the best fit for primary helm use because they provide larger touch targets, better layout stability, easier reading at helm distance, and more room for simultaneous route and app awareness.

1.4.2 Phones

Phones work as companion displays. Expect a denser layout and less room for simultaneous awareness. For close pilotage, recovery operations, or high-workload situations, a larger display is safer.

1.4.3 Desktop systems

On desktop platforms, FairWindSK can be used as a fixed navigation station display. The interface should be treated as a marine operational display first

regardless of input method.

2 Before You Leave the Dock

Run through these checks before every departure. Five minutes at the dock is worth an hour at sea.

2.1 First-Time Setup

When FairWindSK is installed on a new display or freshly configured system, work through these steps before relying on it underway:

1. Open *Settings* > *Connection*. Enter the Signal K server address, e.g. `http://192.168.1.100:30` Tap **Connect**. Watch the status change to *Live*.
2. Open *Settings* > *Units*. Choose knots for speed, nautical miles for distance, metres or feet for depth.
3. Open *Settings* > *Applications*. Verify that the apps you expect are present. Arrange page 1 with the apps you use on every passage.
4. Open *Settings* > *Comfort* and choose the *Day* preset. Verify readability from your helm position.
5. If your installation includes autopilot, anchor, alarms, or other plugin-backed features, confirm that each is available and responds correctly before departure.
6. If the device has no physical keyboard, enable the virtual keyboard in *Settings* > *Main* and restart FairWindSK.
7. Run a short test passage in calm conditions before relying on FairWindSK offshore.

> Do not assume a feature is ready simply because its icon is visible. Confirm live data, control response, and real-world behaviour first.

2.2 Pre-Departure Checklist

☐	Check
☐	FairWindSK starts normally and the shell health state is normal
☐	Signal K connection is healthy (*Live* in the status area)
☐	Position live in Top Bar — latitude and longitude plausible
☐	SOG updating — vessel has a valid GPS fix
☐	Depth updating (if depth sounder is fitted)
☐	Heading showing from compass or GPS
☐	Chart plotter open and vessel in correct position on chart
☐	Route or waypoint active if proceeding to a destination
☐	AIS targets visible in chart plotter (if AIS receiver is fitted)
☐	Display brightness and comfort preset match the current light conditions
☐	Screen is readable and reachable from the normal operating position
☐	POB button location confirmed — know exactly where it is
☐	Autopilot available and responding correctly (if required)
☐	Anchor watch function confirmed available (if anchoring tonight)

Part II

The Interface

3 Screen Layout

FairWindSK keeps three shell areas visible at all times. These names are used consistently throughout this manual, in the project source code, and in all contributor documentation.

3.1 The Three Zones

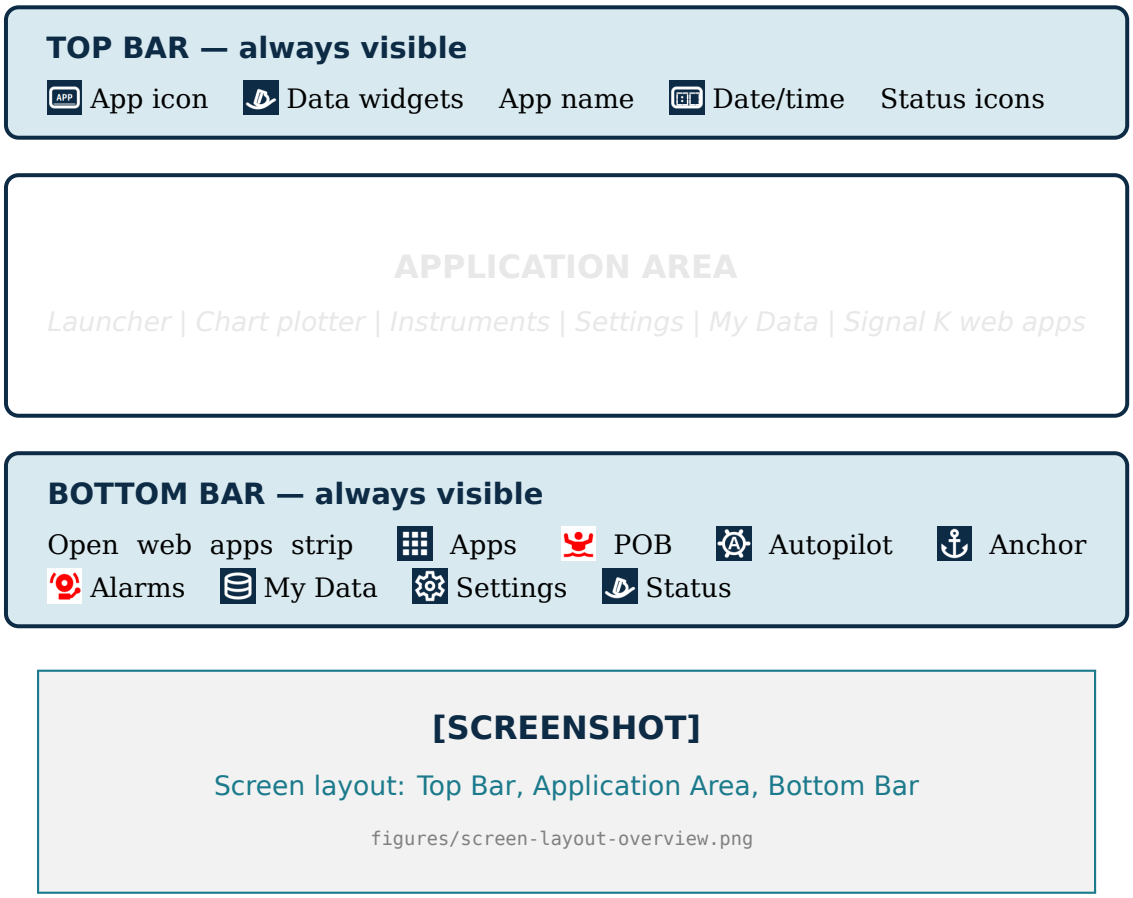

Figure 3.1. Screen layout: Top Bar, Application Area, Bottom Bar

3.2 The Application Area

The Application Area is the main working space between the bars. This is where FairWindSK shows:

- launcher pages
- chart and navigation apps
- Settings
- My Data
- embedded Signal K web applications

The active view must not visually overlap the Top Bar or Bottom Bar unless a formally defined drawer is temporarily covering it.

3.3 Drawers

Drawer	Description
Bottom Bar horizontal drawer	Modal dialog opening upward from the Bottom Bar. Used for web modal dialogs, confirmations, file pickers, and settings sub-panels. Only one dialog is visible at a time; additional dialogs stack. Can extend upward to temporarily cover the Application Area. The hosted UI must fit inside the drawer without requiring vertical scrolling.
Application Area vertical drawer	Non-modal panel opening leftward from the right edge of the Application Area. Used by apps for contextual menus or tool palettes. The Application Area remains interactive behind it.

3.4 Single-Window Model

FairWindSK enforces a single-window design on all platforms. No app opens a second window; no external native application is launched. `file://` launcher entries are blocked.

On desktop platforms (Linux, macOS, Windows, Raspberry Pi OS), press **SHIFT+TAB** to bring FairWindSK back to the foreground if a hosted web application has taken full focus.

4 The Top Bar

The Top Bar is always visible. Its purpose: provide a two-second confidence check at any moment. Use it to answer: *What is happening right now, and is the data still trustworthy?*

[SCREENSHOT]

Top Bar: position, SOG, DPT, BTW, ETA and status icons

`figures/topbar-overview.png`

Figure 4.1. Top Bar: position, SOG, DPT, BTW, ETA and status icons

4.1 Fixed Top Bar Elements

Icon	Element	Function
	App icon	Icon of the current application. Tapping opens the Application Area vertical drawer.
—	App name	Name of the currently active application.
	Date / time	Current device time and date.
	Status icons	REST and stream health indicators.

4.2 Data Widgets — Complete Reference

Each widget subscribes to a Signal K path configured in *Settings > Signal K Paths*.

Icon	Key	Default Signal K path	Displays	Operational significance

L/L	pos	navigation.position	Latitude / longitude	GPS fix confirmation
COG	cog	navigation.courseOverGroundT	COG (°T)	Actual track over seabed
SOG	sog	navigation.speedOverGround	SOG (kn)	True speed for ETAs
HDG	hdg	navigation.headingTrue	Heading (°T)	Where the bow points
STW	stw	navigation.speedThroughWater	STW (kn)	Sail trim, current deduction
DPT	dpt	environment.depth.belowTrans	Depth below transducer	Safety-critical in shoal water
BTW	btw	navigation.course.calcValues	Bearing to WPT	Course to steer
DTG	dtg	navigation.course.calcValues	Distance to go	Passage progress
TTG	ttg	navigation.course.calcValues	Time to go	ETA planning
ETA	eta	navigation.course.calcValues	Estimated arrival	Crew notification
XTE	xte	navigation.course.calcValues	Cross-track error	Route compliance
VMG	vmg	performance.velocityMadeGood	Velocity made good	True sailing efficiency
WPT	wpt	navigation.course.nextPoint	Active waypoint name	Route awareness

Missing depth is **not** zero depth. Missing means the measurement is unavailable — treat it as entirely unknown. If depth becomes stale or missing in shallow or restricted water, verify immediately and adjust speed and manoeuvring margin accordingly.

4.3 Reading Navigation Values Correctly

4.3.1 COG and SOG

COG is course over ground; SOG is speed over ground. These show how the boat is actually moving over the earth, which may differ from where the bow is

pointing due to leeway and tidal current.

4.3.2 Heading

Heading tells you where the bow is pointed. If heading becomes stale or missing, steer conservatively and cross-check with a trusted compass.

4.3.3 Route and waypoint values

When routing is active, BTW, DTG, TTG, ETA, XTE, and VMG all update continuously. If any of these values stop updating, check the chart and your surroundings before relying on the route guidance.

4.4 Configuring the Top Bar

Open *Settings* > *Top Bar*. A live preview of the bar appears at the top; the widget palette is below. Use the ← move-left and → move-right buttons to reposition widgets. Toggle **Show Icon**, **Show Text**, **Show Units**, and **Show Trend** per widget. Tap ↻ **Reset Defaults** to restore the built-in layout.

> In rough conditions, a lean Top Bar with 3–4 key widgets (position, SOG, DPT, BTW) is safer than twelve items. Add widgets only if they add genuine situational awareness at the current passage phase.

5 The Bottom Bar

The Bottom Bar is the permanent action strip at the bottom of every screen. Its spatial stability is intentional: the Apps button is always in the same place, the POB button is always within reach. Muscle memory is safety.

[SCREENSHOT]

Bottom Bar: open-apps strip, main icons, Signal K status

figures/bottombar-overview.png

Figure 5.1. Bottom Bar: open-apps strip, main icons, Signal K status

5.1 Bottom Bar Regions

Region	Description
Open web-apps strip	Scrollable row of small app icons on the left, one per loaded application. Tap any icon to jump to that application instantly. The active application is highlighted.
Main icons	The central group of seven fixed icons. Spatially stable across all screens and comfort presets.
Signal K status area	Right side. Compact stream and REST health indicators showing server name, last update timestamp, and connection state. Tapping opens a status detail drawer.

5.2 Main Icon Reference

These are the exact icons defined in `BottomBar.ui`.

Icon	Label	Function	Notes
	Apps	Opens the launcher. Hides the current application and shows the tile grid.	Use as the safe escape hatch from any stuck or confusing screen.
	POB	Person Over Board. One tap starts the emergency.	Always visible. Never disabled.
	Autopilot	Opens the Autopilot bar.	Grey/inactive if no compatible autopilot plugin is detected.
	Anchor	Opens the Anchor Watch bar.	Grey/inactive if no anchor plugin is detected.
	Alarms	Opens the Alarms bar. Highlighted when any alarm is active.	Accent colour when active; draws the eye across all presets.
	My Data	Opens the resource manager (Chapter 12).	
	Settings	Opens all configuration (Chapter 20).	

5.3 Operational Overlay Bars

Tapping POB, Autopilot, Anchor, or Alarms replaces the normal Bottom Bar icons with a dedicated operational bar. The Application Area continues running behind the bar. Tap close (rightmost icon) to return to the normal Bottom Bar without changing any operational state.

> Operational bars are not modal. Data continues updating. The chart plotter and all applications remain live behind the operational bar.

6 The Launcher and Application Management

The launcher is the safe home position of the shell. Use it to return to app tiles, switch between tasks, leave a frozen app, or reorient yourself after a confusing workflow. Whenever the current screen stops feeling clear or trustworthy, tap **Apps** to return here.

[SCREENSHOT]

Launcher home screen: tile grid, page indicator, Bottom Bar

`figures/launcher-overview.png`

Figure 6.1. Launcher home screen: tile grid, page indicator, Bottom Bar

6.1 Opening an Application

1. Tap **Apps** in the Bottom Bar.
2. Navigate pages by swiping if needed.
3. Tap the tile for the application you want.
4. Wait for it to load in the Application Area.

If an app does not appear correctly, avoid repeated rapid taps. Wait briefly, retry once if the shell offers the option, then return to the launcher if the app still looks wrong.

6.2 Switching Between Open Applications

Use the open-apps strip in the Bottom Bar to switch between loaded applications with a single tap. The strip shows a small icon for each loaded application; the active one is highlighted.

6.3 Adding a Custom Application

1. Open *Settings* > *Applications*.
2. Tap **Create app**.
3. Enter a display name and the full URL (`http://` or `https://`).
4. Optionally set a custom icon path and zoom level.
5. Tap **Save**.

6.4 Recovering a Stuck or Blank Application

1. Wait briefly. Retry once if the shell offers a retry action.
2. Tap **Apps** to return to the launcher.
3. Check shell health and connection state.
4. Reopen the app only after confirming the shell itself is healthy.

> A failed application does not mean FairWindSK has failed. The shell, data widgets, and operational bars (POB, anchor, autopilot) continue working independently of any individual application. A degraded app is not the same as a degraded vessel state.

Part III

Operational Features

7 Signal K and Data Health

At sea, a number is only useful if you know whether it is still fresh. FairWindSK uses three data states consistently throughout the interface and operational documentation.

7.1 Individual Value States

State	Meaning	Recommended action
Live	Value received recently and updating normally.	Use normally. Continue to glance at the Top Bar periodically.
Stale	Updates have slowed or stopped. Last known value shown, dimmed. A warning, not current truth.	Treat with caution. Cross-check with independent instruments. Investigate if the state persists.
Missing	No usable value. A dash or placeholder is shown. **Missing is not zero.** It means unavailable.	Treat as entirely unknown. Never use as a safe default. Verify with another source.

7.2 Connection Health States

State	Meaning	Response
Live	Fully healthy.	Continue normally.
Connecting	Establishing the initial connection.	Wait 30–60 seconds.

Reconnecting	Connection lost; auto-recovery in progress.	Wait briefly. Usually resolves automatically.
Stale	Connected but updates slowed or delayed.	Cross-check instruments. Investigate if it persists.
REST degraded	Request/response channel impaired.	Settings pages may fail; monitor closely.
Stream degraded	Live stream impaired.	Readouts may stop updating; treat as stale.
Disconnected	No connection.	Check Signal K server power and Wi-Fi.
Apps loading	App catalogue being fetched.	Wait; apps appear when done.
Apps stale	App catalogue may be out of date.	Apps still work; list may be incomplete.
Foreground degraded	Current application has a problem.	Retry once; if still failing, return to launcher and check shell health.

7.3 When to Act Immediately

Act without delay when:

- depth becomes stale or missing in shallow or restricted water
- position or heading disappears during close-quarters navigation
- a POB event occurs
- an autopilot command does not match the vessel’s actual response

Monitor briefly and then cross-check when:

- the shell is reconnecting after a brief drop
- app loading takes longer than expected
- route guidance values stop updating
- only one app looks degraded while the rest of the shell is healthy

7.4 Automatic Recovery Cycle

When a connection is lost, FairWindSK automatically:

1. Detects the disconnection or degradation.
2. Performs a fresh Signal K server discovery (handles endpoint changes after a restart).
3. Re-establishes the WebSocket stream.
4. Re-sends all active subscriptions.
5. Performs an immediate REST snapshot so widgets show current values right away.
6. Refreshes the app catalogue and all My Data resource models.
7. Refreshes unit preferences from the server.

When a Signal K server restart is triggered from inside FairWindSK (via the Signal K admin web app), FairWindSK detects the planned restart, pauses its reconnect loop briefly, and then resumes recovery automatically.

After any reconnect or server restart, wait for the recovery to complete and confirm that live data returns before depending on safety-critical values.

> After a power interruption, GPS receivers and instruments take time to re-acquire and settle. Even after FairWindSK shows *Live*, verify each safety-critical value before acting on it.

8 Person Over Board (POB)

Mark a person overboard event immediately. Do not delay because you expect to remember the position later. Mark first, then continue recovery actions. The POB function assists with recovery reference. It does not replace immediate crew response, lookout, helmsmanship, or established emergency procedures.

8.1 Activating POB

Tap **POB** in the Bottom Bar. It is permanently visible and never disabled. Practise finding it in the dark and in rough conditions.

In the instant you tap POB, FairWindSK simultaneously:

- Captures the current GPS position as the incident position.
- Creates a timestamped waypoint resource on Signal K named `POB HH:MM:SS` (UTC).
- Sets that waypoint as the active navigation destination — all navigation apps immediately show bearing and distance to the POB.
- Raises the standard Signal K `notifications.mob` alarm at emergency level, alerting all connected devices and displays.
- Starts the elapsed-time clock counting up every second.
- Subscribes to live bearing and distance from the vessel to the POB waypoint.
- Replaces the normal Bottom Bar with the POB dialog.

[SCREENSHOT]

POB Bar: bearing, distance, elapsed time, SAR search buttons

`figures/pob-bar-active.png`

Figure 8.1. POB Bar: bearing, distance, elapsed time, SAR search buttons

8.2 POB Bar Controls and Readouts

Icon	Control	Description
—	POB selector	Drop-down listing all active POB events. Select an event to display its data.
—	Last known position	Latitude and longitude at the moment POB was pressed, in the configured coordinate format.
—	Bearing	Live compass bearing from vessel to POB waypoint. Updates continuously as the vessel manoeuvres.
—	Distance	Live distance from vessel to POB waypoint. Updates continuously.
—	Elapsed time	HH:MM:SS counting up from the moment POB was pressed.
—	Cancel	Cancels the selected incident. Removes the Signal K alarm and navigation destination for that event.
	Spiral search	Generates a SAR spiral route centred on the POB position.
	Parallel search	Generates a SAR parallel-track route centred on the POB position.
	Close dialog	Returns to the normal Bottom Bar. **The alarm and waypoint remain active on Signal K.**

Tapping Close does **not** cancel the POB. The Signal K alarm and waypoint remain active until explicitly cancelled. Only the **Cancel** button removes a specific incident.

8.3 SAR Search Patterns

8.3.1 Spiral Search

Tap **Spiral**. FairWindSK creates a 500×500 m SAR area region centred on the POB position and a spiral route starting at the POB and radiating outward

in 50 m spacing legs. Both the region and route are saved as Signal K resource objects and appear immediately in *My Data* > *Regions* and *My Data* > *Routes*.

8.3.2 Parallel-Track Search

Tap **Parallel**. FairWindSK creates the same 500×500 m area region and a creeping-line parallel-track route covering it in 60 m-spaced parallel legs.

8.4 Cancelling a POB

1. Select the incident in the POB selector (if multiple incidents are active).
2. Tap **Cancel**.
3. FairWindSK removes the Signal K alarm and clears the navigation destination.
4. If no more active incidents remain, the POB dialog closes automatically.

> The POB waypoint is **not** deleted when cancelled. It is retained in *My Data* > *Waypoints* with its timestamp and position for accident reporting and post-incident debrief.

9 Autopilot Controls

Tap **Autopilot** in the Bottom Bar to open the Autopilot bar, which replaces the normal Bottom Bar. The Autopilot icon is active only when the required autopilot plugin is installed and FairWindSK can interact with it.

> Always verify the real-world effect of an autopilot command on the vessel. Never assume a mode change took effect because you tapped a control. Be prepared to disengage or override immediately if the response is unsafe or unexpected.

[SCREENSHOT]

Autopilot Bar: mode buttons, trim controls, rudder indicator

`figures/autopilot-bar.png`

Figure 9.1. Autopilot Bar: mode buttons, trim controls, rudder indicator

9.1 Autopilot Bar Controls

Icons are the exact assets defined in `AutopilotBar.cpp`.

Icon	Control	Signal K action	When to use
	Stand By	Disengage	Any time you want manual control. Verify the helm responds.
	Auto	Compass mode	Open-water passages; holds current heading.
	Wind	Wind-vane mode	Upwind/downwind; holds wind angle.
	Route	GPS route mode	Following a waypoint route.

−	$-1°$ port	Trim $-1°$	Fine adjustment to port.
+	$+1°$ stbd	Trim $+1°$	Fine adjustment to starboard.
«	$-10°$ port	Adjust $-10°$	Larger correction to port.
»	$+10°$ stbd	Adjust $+10°$	Larger correction to starboard.
←	Tack port	Tack to port	Sailing upwind.
→	Tack stbd	Tack to starboard	Sailing upwind.
«	Gybe port	Gybe to port	Light–moderate conditions; monitor.
»	Gybe stbd	Gybe to starboard	Supervise at helm in strong winds.
○	Next WPT	Advance route leg	Route navigation.
🔔	Dodge	Obstacle avoidance	Lobster pot, vessel, hazard.

9.2 Rudder Indicator

The Autopilot bar shows a **read-only** horizontal slider displaying current rudder deflection from $-30°$ (full port) to $+30°$ (full starboard). This reflects the actual rudder position reported by the pilot — not a commanded heading. Use it to confirm the autopilot is responding to commands as expected.

10 Anchor Watch

Tap **Anchor** to open the Anchor Watch bar.

> Electronic anchor watch is an aid only. In an exposed anchorage, deteriorating weather, or poor holding ground, post a human watch regardless of what any electronic device shows.

[SCREENSHOT]

Anchor Watch Bar: position, depth, rode, radius, windlass controls

`figures/anchor-bar.png`

Figure 10.1. Anchor Watch Bar: position, depth, rode, radius, windlass controls

10.1 Anchor Bar Controls and Readouts

Icons are the exact assets defined in `AnchorBar.ui`.

Icon	Control	Description
	Drop	Records GPS position as anchor point and starts monitoring.
	Raise	Stops monitoring and clears the anchor position.
	Release	Issues a release-rode command to an electric windlass.
	Up (hold)	Continuous haul-in command to windlass. Release to stop.
	Down (hold)	Continuous let-out command to windlass. Release to stop.
	Reset	Resets anchor position to current GPS position.

—	Radius slider	Maximum distance permitted before alarm.
—	Set Radius	Commits the radius and sends it to the anchor plugin.
—	LAT / LON	GPS coordinates of the recorded anchor position.
—	Depth	Current water depth from the depth sounder.
—	Rode	Length of rode deployed.
—	Current radius / bearing	Live distance and bearing from vessel to anchor.
—	Max radius	Maximum distance reached since watch started.

10.2 Setting Anchor Watch — Step by Step

1. Anchor normally and pay out your intended scope.
2. After the vessel has settled, tap in the Bottom Bar.
3. Confirm the displayed GPS position is where the anchor is lying.
4. Tap **Drop**. FairWindSK records the position and starts monitoring.
5. Set the radius, allowing for scope, swing circle, GPS accuracy (± 3–10 m), and a safety margin.
6. Tap **Set Radius** to commit.
7. Close the bar. Monitoring continues in the background.

11 Alarms

Tap **Alarms** to open the Alarms bar. When any alarm is active the icon is highlighted with accent colouring across all comfort presets — designed to catch the eye even during the night watch.

[SCREENSHOT]

Alarms Bar: SOLAS emergency buttons and active alarm list

`figures/alarms-bar.png`

Figure 11.1. Alarms Bar: SOLAS emergency buttons and active alarm list

11.1 Emergency Alarm Buttons

Icons are the exact assets defined in `AlarmsBar.ui`.

Icon	Button	Signal K path	Emergency type
	POB	`notifications.mob`	Person / man overboard
	Sinking	`notifications.sinking`	Vessel taking water or sinking
	Fire	`notifications.fire`	Fire aboard
	Piracy	`notifications.piracy`	Piracy or armed attack
	Abandon	`notifications.abandon`	Abandon ship
	Adrift	`notifications.adrift`	Vessel adrift and uncontrolled

Activating an alarm in FairWindSK raises a Signal K notification on the onboard network only. It does **not** contact coast guard, MRCC, or any external authority. Make a DSC and/or voice distress call on VHF Channel 16 in any real emergency.

11.2 Alarm Priority and Response

Handle alerts in this order:

1. Safety-critical values first — depth, position, heading.
2. Connection and shell health second.
3. App-specific problems third.

11.3 Active Alarm List

Below the buttons, the Alarms bar lists all currently active Signal K notifications, showing path, state (emergency/alarm/warn), message text, and timestamp. It also shows the current Signal K connection state and the last stream update timestamp — a compact health summary without leaving the bar.

Part IV

My Data

12 My Data — Overview

My Data is FairWindSK's built-in resource manager. It provides a dedicated, touch-optimised interface for all navigational data stored on the Signal K server: waypoints, routes, regions, notes, charts, track history, and files.

Open My Data by tapping the **My Data** icon (≡) in the Bottom Bar, or by tapping its tile on the launcher.

[SCREENSHOT]

My Data — tab bar showing all seven tabs

figures/mydata-main-tabs.png

Figure 12.1. My Data — tab bar showing all seven tabs

All resources in My Data are stored on the Signal K server, not on the FairWindSK device. They are accessible from any device connected to the same Signal K server simultaneously.

12.1 My Data Tab Overview

My Data is organised into seven tabs. Each tab loads lazily — it is created only the first time it is tapped, keeping startup fast.

Tab	Name	Contents
0	Waypoints	Named positions: anchorages, hazards, marinas, fishing marks, race marks.
1	Routes	Ordered sequences of waypoints forming planned passages.
2	Regions	Geographic boundary areas — SAR search regions, exclusion zones, anchorage limits.
3	Notes	Free-text annotations attached to positions.

4	Charts	Chart tile source catalogue for installed chart providers.
5	Tracks	Historical GPS track — the vessel's recorded position history.
6	Files	Direct filesystem browser for files accessible through Signal K.

12.2 The Common Toolbar

Tabs 0–4 (Waypoints, Routes, Regions, Notes, Charts) share a common interface layout:

- A **title label** showing the resource type.
- A **search field** for real-time filtering of the list. Searching is case-insensitive and checks all visible columns simultaneously.
- A **scrollable table** showing all resources from the Signal K server. Rows are 64 px tall for comfortable touch selection. Alternating row colours aid readability. Column headers are clickable for sorting.
- A **toolbar** of icon buttons:

Button	Action
Open (→)	Opens the selected resource — shows its details, opens an embedded map preview (Waypoints), or views the attachment (Notes). Accent-coloured; primary action.
Edit (□)	Opens the resource editor drawer for the selected resource.
New (+)	Opens the resource editor drawer to create a new resource.
Delete (□)	Deletes the selected resource after confirmation. Cannot be undone.
Import	Imports resources from a GeoJSON or JSON file. File picker opens in the Bottom Bar drawer.
Export	Exports the selected resource (or all resources if none selected) to a GeoJSON file.

Refresh	Forces a reload of all resources from the Signal K server.

On Raspberry Pi OS (ARM Linux), the toolbar uses text-only buttons to avoid icon rendering issues on that platform. The button layout and functionality are identical; only the visual presentation differs.

13 Waypoints

Waypoints are named positions stored on the Signal K server. They are used for anchorages, hazards, marinas, fuel stops, fishing marks, race marks, and any other position you want to record and navigate to.

[SCREENSHOT]

My Data — Waypoints tab, list view with name, description, type, lat/lon, timestamp

`figures/mydata-waypoints-list.png`

Figure 13.1. My Data — Waypoints tab, list view with name, description, type, lat/lon, timestamp

13.1 Waypoints Table Columns

The waypoints table shows six columns:

Column	Description
Name	Display name of the waypoint, taken from the `name` field or from `feature.properties.name` in the GeoJSON resource.
Description	Free-text description of the waypoint.
Type	The waypoint type tag, e.g. `alarm-mob` (POB waypoints created by FairWindSK), `anchor`, `marina`, or any custom type. POB waypoints are visually distinguishable by their type.
Latitude	Latitude in your configured coordinate format (right-aligned).
Longitude	Longitude in your configured coordinate format (right-aligned).
Timestamp	ISO 8601 creation or last-update timestamp.

13.2 Searching Waypoints

The search field above the table filters waypoints by any visible column. Type part of a name, description, type, or coordinate value. The table updates in real time as you type.

13.3 Creating a Waypoint

Tap **New** to open the Waypoint Editor drawer.

[SCREENSHOT]

Waypoint Editor — name, description, type, latitude/longitude, altitude fields

`figures/mydata-waypoint-editor.png`

Figure 13.2. Waypoint Editor — name, description, type, latitude/longitude, altitude fields

13.3.1 Editor Fields

Field	Description
Name	The display name shown in the table, Top Bar waypoint widget, and chart plotter.
Description	Free-text notes about this position.
Type	Optional type tag. Leave blank for a generic waypoint. Use `anchor` for anchoring positions, `hazard` for obstructions, etc.
Latitude	Degrees with decimal minutes or decimal degrees, depending on the active coordinate format. Range: -90 to $+90$.
Longitude	Range: -180 to $+180$.
Altitude	Altitude in metres (optional). Leave as 0 if not relevant.

After filling in the fields, tap **OK** in the editor. The waypoint is created immediately on the Signal K server and appears in the table.

13.4 Editing a Waypoint

Select a waypoint in the table, then tap **Edit**. The same editor drawer opens with the existing values pre-filled. Modify any field and tap **OK**. The change is written immediately to Signal K.

13.5 Deleting a Waypoint

Select a waypoint and tap **Delete**. A confirmation drawer appears. Tap **Delete** to confirm. The waypoint is removed from the Signal K server permanently.

> Deletion cannot be undone. POB waypoints (type `alarm-mob`) should generally be retained for accident reporting and debrief, not deleted.

13.6 Opening a Waypoint — the Detail View

Select a waypoint and tap **Open** (or double-tap the row). The Waypoint Detail view opens.

[SCREENSHOT]

Waypoint detail — name, description, position, Freeboard-SK map preview

`figures/mydata-waypoint-detail.png`

Figure 13.3. Waypoint detail — name, description, position, Freeboard-SK map preview

The detail view shows:

- Full name, description, type, and formatted coordinates.
- An embedded Freeboard-SK map preview centred on the waypoint's position, if Freeboard-SK is installed on the connected Signal K server. The map uses a JavaScript function to compute the view bounding box from the waypoint coordinates and zoom level.
- A **Navigate** button to set the waypoint as the active navigation destination.

> The embedded Freeboard-SK preview is disabled on Raspberry Pi OS (ARM Linux) builds to avoid performance issues with simultaneous WebEngine instances on that platform. Coordinates and descriptions are always shown in text form regardless.

13.7 Navigating to a Waypoint

From the detail view, tap **Navigate**. FairWindSK calls `navigateToWaypoint` on the Signal K client, which writes the waypoint's resource path (e.g. `/resources/waypoints/\{uuid\}`) as the active navigation destination on Signal K. Immediately:

- The Top Bar BTW, DTW, TTG, and ETA widgets begin showing values relative to this waypoint.
- Freeboard-SK and any other navigation application subscribed to `navigation.course` show the destination.
- The POB bearing and distance displays (when the POB bar is active) use this same destination.

13.8 Importing Waypoints

Tap **Import**. A file picker opens in the Bottom Bar drawer. Select a GeoJSON file containing `Feature` objects with `Point` geometry.

FairWindSK:

1. Reads the GeoJSON file.
2. Extracts each `Point` feature as a waypoint.
3. Creates or updates each waypoint on the Signal K server (update if a matching UUID is found; create if not).
4. Reports the count of imported waypoints in an informational drawer.
5. Selects the first imported waypoint in the table.

The import file filter accepts `.geojson` and `.json` files.

13.9 Exporting Waypoints

Tap **Export**. If a waypoint is selected, only that waypoint is exported. If nothing is selected, all waypoints are exported. A file-save picker opens. Choose a filename and location. The output is a valid GeoJSON `FeatureCollection` with `Point` geometry features.

14 Routes

Routes are ordered sequences of waypoints forming planned passages. They are typically created in a chart plotter (Freeboard-SK) and stored on the Signal K server.

[SCREENSHOT]

My Data — Routes tab, list view with name, geometry type, point count, timestamp

`figures/mydata-routes-list.png`

Figure 14.1. My Data — Routes tab, list view with name, geometry type, point count, timestamp

14.1 Routes Table Columns

Column	Description
Name	Display name of the route.
Description	Free-text description or summary.
Geometry	The GeoJSON geometry type — typically `LineString` for simple routes, `MultiLineString` for multi-leg routes with gaps, or `sar-spiral` / `sar-parallel` for SAR patterns created by the POB feature.
Points	The number of coordinate points in the route geometry. Displayed centred.
Timestamp	ISO 8601 creation or last-update timestamp.

14.2 Opening a Route — the Detail View

Select a route and tap **Open**.

[SCREENSHOT]

Route detail — name, description, waypoint list, GeoJSON preview

`figures/mydata-route-detail.png`

Figure 14.2. Route detail — name, description, waypoint list, GeoJSON preview

The detail view shows:

- Name, description, geometry type, and point count.
- A GeoJSON geometry preview if available.
- An **Activate** button to set the route's first waypoint as the active navigation destination.

14.3 Creating and Editing Routes

14.3.1 Creating a Route

Tap **New**. The Route Editor drawer opens.

Field	Description
Name	Display name of the route.
Description	Free-text notes.
Coordinates (JSON)	The GeoJSON coordinate array for the route geometry. Enter as a JSON array of `[longitude, latitude]` pairs.
Geometry (JSON)	Advanced: edit the full GeoJSON geometry object if needed.
Properties (JSON)	Advanced: additional GeoJSON feature properties.

14.3.2 Activating a Route

From the route detail view, tap **Activate**. FairWindSK sets the first waypoint of the route as the active Signal K navigation destination. The Autopilot Route mode (Chapter 9) will then follow the active route automatically.

14.4 Importing and Exporting Routes

Import and export use the same GeoJSON `FeatureCollection` format as Waypoints. The geometry type in the GeoJSON determines how the route is stored on Signal K.

> SAR search routes created by the POB feature (Chapter 8) appear in the Routes tab automatically after creation. They have geometry types `sar-spiral` and `sar-parallel` and can be exported as GeoJSON for voyage records.

15 Regions

Regions are geographic boundary areas defined as GeoJSON polygons. They are used for SAR search areas (automatically created by the POB feature), marine protected area boundaries, exclusion zones, anchorage limits, and fishing grounds.

[SCREENSHOT]

My Data — Regions tab, list view with name, geometry type, timestamp

figures/mydata-regions-list.png

Figure 15.1. My Data — Regions tab, list view with name, geometry type, timestamp

15.1 Regions Table Columns

Column	Description
Name	Display name of the region.
Description	Free-text description.
Geometry	The GeoJSON geometry type — typically `Polygon` or `MultiPolygon`. SAR regions use `Polygon`.
Timestamp	ISO 8601 creation or last-update timestamp.

15.2 Creating a Region

Tap **New**. The Region Editor drawer opens.

Field	Description
Name	Display name.

Description	Free-text notes.
Coordinates (JSON)	The GeoJSON coordinate array for the polygon. Polygons require an outer ring (first and last coordinate identical to close the ring).
Geometry (JSON)	Advanced: full GeoJSON geometry object.
Properties (JSON)	Advanced: additional GeoJSON feature properties.

SAR search areas created by the POB feature appear in the Regions tab automatically. Their `properties` include the `pobUuid` linking them to the originating POB incident.

16 Notes

Notes are free-text annotations attached to geographic positions. They function as sticky notes on the chart — useful for recording pilotage tips, local hazards not on the chart, marina details, or crew observations.

[SCREENSHOT]

My Data — Notes tab, list view with title, description, href, MIME type, timestamp

`figures/mydata-notes-list.png`

Figure 16.1. My Data — Notes tab, list view with title, description, href, MIME type, timestamp

16.1 Notes Table Columns

Notes have a distinct column set reflecting their attachment-oriented nature:

Column	Description
Title	The display name of the note (the first column is labelled *Title* for notes, not *Name*, to reflect the note-taking context).
Description	Summary text. For imported text files, this is set to the first 120 characters of the file content.
Href	An optional file path or URL linking the note to an attachment — a PDF pilot book page, an image, a text file. When present, tapping **Open** opens the attachment directly in the built-in viewer.
MIME Type	The MIME type of the attachment, e.g. `application/pdf`, `image/jpeg`, `text/plain`. Used to route the file to the correct viewer.
Timestamp	ISO 8601 creation or last-update timestamp.

16.2 Creating a Note

Tap **New**. The Note Editor drawer opens.

[SCREENSHOT]

Note Editor — title, description, href, MIME type, position fields

`figures/mydata-note-editor.png`

Figure 16.2. Note Editor — title, description, href, MIME type, position fields

Field	Description
Title	The note title (displayed in the *Title* column).
Description	Main text content of the note.
Href	Optional path or URL to an attachment file. Enter an absolute path (e.g. `/home/pi/charts/harbour.pdf`) or a URL.
MIME Type	Optional MIME type for the attachment. FairWindSK uses this to select the correct viewer. If left blank, FairWindSK detects the type from the file extension.
Has position	Check this box to attach a geographic position to the note.
Latitude	Latitude when *Has position* is checked.
Longitude	Longitude when *Has position* is checked.

16.3 Opening a Note

16.3.1 Notes Without an Attachment

Tapping **Open** opens the note editor in read-only mode, showing the title, description, and position.

16.3.2 Notes With an Attachment (href set)

Tapping **Open** immediately opens the attachment:

- **Local file that exists:** opens in the built-in FileViewer (PDF, image, or text viewer depending on the MIME type).

- **HTTP/HTTPS URL:** opens in an embedded web view inside the Bottom Bar horizontal drawer.
- **Local directory path:** shows a message directing you to use My Data > Files to browse the folder (single-window model prevents opening a separate file manager).

16.4 Importing Notes

The Notes tab accepts a broader set of import file types than other tabs. In addition to GeoJSON/JSON:

- **Text files** (`.txt`, `.md`, `.html`): FairWindSK creates a note with the filename as the title and the first 120 characters of the file content as the description, setting the `href` to the absolute file path and detecting the MIME type automatically.
- **Images** (`.jpg`, `.jpeg`, `.png`): Creates a note with the filename as title and sets the `href` and `mimeType` accordingly.
- **PDF files**: Creates a note pointing to the PDF file via `href`.

The import filter text reads: *Notes and attachments (*.geojson *.json *.txt *.md *.markdown *.html *.htm *.pdf *.jpg *.jpeg *.png)*.

17 Charts

The Charts tab lists available chart tile sources registered on the Signal K server. It manages the chart source catalogue — not the actual chart display, which happens inside Freeboard-SK or another chart plotter running in the Application Area.

[SCREENSHOT]

My Data — Charts tab, list view with name, format, identifier, URL, timestamp

`figures/mydata-charts-list.png`

Figure 17.1. My Data — Charts tab, list view with name, format, identifier, URL, timestamp

17.1 Charts Table Columns

Column	Description
Name	Display name of the chart source.
Description	Free-text description.
Format	The chart tile format. Supported formats include: `mbtiles` (MBTiles SQLite database), `kap` (BSB/KAP raster charts), `gemf` (GEMF tile archive), `tilemapresource` (TileMapResource XML), `package` (ZIP chart package).
Identifier	The chart identifier used by Signal K and chart plotters to reference this source.
Chart URL	The local filesystem path or URL where the chart tiles are served.
Timestamp	ISO 8601 registration timestamp.

17.2 Adding a Chart Source

Tap **New**. The Chart Editor drawer opens.

Field	Description
Name	Display name for this chart source.
Description	Free-text description.
Identifier	Unique identifier string used by chart plotters to reference this source.
Chart format	The format of the chart tiles. FairWindSK detects the format from the file extension when importing.
Chart URL	Path or URL to the chart tiles. For MBTiles, this is the absolute path to the `.mbtiles` file.
Tilemap URL	For `tilemapresource` format only: the path to the `tilemapresource.xml` file. The Chart URL is then set to the directory containing it.
Chart region	Optional: a GeoJSON bounding box or geometry describing the geographic coverage of this chart.
Chart scale	Optional: the chart scale denominator (e.g. `50000` for 1:50,000).
Chart layers	Optional: comma-separated list of layer names for tile sources that support multiple layers.
Chart bounds	Optional: GeoJSON coordinates describing the chart bounds.

17.3 Importing a Chart Source

Tap **Import**. Select a chart file. FairWindSK detects the chart format from the file extension and constructs a chart resource object automatically. For `tilemapresource.xml` files, FairWindSK sets both the tile URL (the directory) and the tilemap URL (the XML file path).

The import creates or updates the chart record on the Signal K server.

17.4 Exporting the Chart Catalogue

Tap **Export**. The chart catalogue is exported as a JSON manifest document with structure:

```
{
```

```
  "type": "FairWindSKChartPackage",
  "charts": [ { ... }, { ... } ]
}
```

This manifest can be imported on another FairWindSK installation to replicate the chart catalogue configuration (it describes chart sources, not the chart tile data itself).

18 Tracks — History

The Tracks tab shows the historical GPS track of the vessel — the recorded sequence of position fixes over time. Tracks are generated automatically by Signal K when the tracks plugin is enabled.

[SCREENSHOT]

My Data — Tracks tab, table of track points with time, lat/lon, altitude

`figures/mydata-tracks-list.png`

Figure 18.1. My Data — Tracks tab, table of track points with time, lat/lon, altitude

18.1 Track Table Columns

The track table shows individual position fixes (points) in chronological order:

Column	Description
Time	The timestamp of the position fix, formatted in local time using the system locale.
Latitude	Latitude to 6 decimal places.
Longitude	Longitude to 6 decimal places.
Altitude	Altitude in metres (if available from the GPS fix; otherwise shown as 0).

18.2 Live Track Recording

If Signal K is actively providing position updates, new track points are appended to the table in real time. The table scrolls to show the most recent point automatically.

18.3 Viewing a Track Point

Select a row and tap **Open**. A read-only dialog opens showing the timestamp, latitude, longitude, and altitude for that point in full detail.

18.3.1 Editing a Track Point

Select a row and tap **Edit**. The Track Point Editor opens.

Field	Description
Timestamp	Date and time of the fix. Uses a date/time picker with up/down arrows.
Latitude	Latitude (8 decimal places, range -90 to $+90$).
Longitude	Longitude (8 decimal places, range -180 to $+180$).
Altitude	Altitude in metres.

18.4 Adding a Track Point Manually

Tap **New**. The same editor opens with default values (current time and position 0,0). Enter the correct values and tap **OK**. The point is inserted into the track in chronological order.

18.5 Deleting a Track Point

Select a row and tap **Delete**. Confirm when prompted. The point is removed from the track.

18.6 Exporting the Track

Tap **Export** to save the complete track as a GeoJSON `LineString` or `FeatureCollection`:

```
{
  "type": "FeatureCollection",
  "features": [
    {
      "type": "Feature",
      "geometry": {
        "type": "LineString",
        "coordinates": [[lon, lat, alt], ...]
```

```
      }
    }
  ]
}
```

The default export filename is `tracks-history.geojson`.

> Export the track after every passage and store it with the voyage logbook. The GeoJSON file can be imported into most GIS tools, chart plotters, and voyage-planning software for review and analysis.

18.7 Importing a Track

Tap **Import** and select a GeoJSON file containing a `LineString` geometry. FairWindSK extracts each coordinate in the line as a track point with a synthetic timestamp, appending them to the current track in order.

18.8 Uses of the Track Tab

- **Passage review:** compare the actual track against the planned route. Identify deviations, traffic avoidance manoeuvres, and current effects.
- **Anchorage assessment:** see the swing circle after anchoring overnight.
- **Safety investigation:** reconstruct the vessel's path before and during an incident for formal reporting.
- **Logbook cross-reference:** correlate positions at specific times with logbook entries.

19 Files

The Files tab is a direct filesystem browser for files and directories accessible on the device running FairWindSK. It provides navigation, search, viewing, and file management operations (copy, cut, paste, rename, delete, new folder) without leaving the FairWindSK interface.

[SCREENSHOT]

My Data — Files tab, directory browser with toolbar

`figures/mydata-files-browser.png`

Figure 19.1. My Data — Files tab, directory browser with toolbar

19.1 Interface Layout

The Files tab is divided into three areas:

- **Toolbar** at the top: navigation and action buttons, path bar, and search field.
- **File list** in the centre: the current directory contents as a tree view with columns for Name, Size, and Date. Row height is 58 px for comfortable touch selection. Alternating row colours aid readability.
- **Actions row** below the toolbar: file operation buttons that appear when items are selected.

19.2 Navigation

19.2.1 Directory Navigation

Action	How to trigger
Enter a folder	Double-tap a folder row in the file list, or select and tap **Open**.

Go up one level	Tap ↑ (Up) button in the toolbar.
Navigate by path	Tap the path bar at the top of the toolbar, type an absolute path, and press Return.
Go home	Tap **Home** button. Returns to the home directory of the running user.

19.2.2 Column Sorting

Tap any column header to sort by that column. The sort direction toggles between ascending and descending on subsequent taps. The default sort is by Name, ascending.

19.3 Searching Files

19.3.1 Basic Search

Tap **Search** or press Return in the search field. FairWindSK recursively searches the current directory tree for files and subdirectories whose names contain the search text.

Results are shown in a flat list in the file view. Tap a result to select it; double-tap to open. The search continues to the root of the browsed directory tree.

[SCREENSHOT]

Files tab — search results view with matched filenames

`figures/mydata-files-search.png`

Figure 19.2. Files tab — search results view with matched filenames

19.3.2 Search Options

Tap **Options** (or the filter button) to expand the search options:

Option	Description
Case Sensitive	When enabled, the search matches the exact capitalisation of the search text. Default: off (case-insensitive).

Search Hidden	When enabled, hidden files and directories (names beginning with `.`) are included in search results. Default: off.
Search System	When enabled, system files and directories (e.g. `/proc`, `/sys` on Linux) are included. Default: off.

The search options button label updates to show a compact summary of active options (e.g. *Options: case, hidden*).

19.4 File Actions

19.4.1 Opening Files

Select a file and tap **Open**, or double-tap the row.

FairWindSK detects the file type using MIME type detection and routes it to the appropriate viewer:

File type	Viewer
Images (`.jpg`, `.jpeg`, `.png`, `.gif`)	Built-in image viewer with pinch-to-zoom and pan.
PDF (`.pdf`)	Built-in PDF viewer, page by page.
Text files (`.txt`, `.md`, `.log`, `.html`)	Built-in text viewer, formatted as plain text or HTML.
Other file types	A message is shown; the file cannot be pre-viewed but can be managed.

Directories open in the directory browser (navigating into them) rather than in a viewer. Attempting to open a directory path from a Notes href shows a message directing you to use the Files tab to browse it instead (single-window model constraint).

[SCREENSHOT]

File Viewer — PDF opened inline within FairWindSK

`figures/mydata-file-viewer.png`

Figure 19.3. File Viewer — PDF opened inline within FairWindSK

19.4.2 Renaming Files and Folders

Select an item and tap **Rename**. An input dialog opens in the Bottom Bar horizontal drawer. Enter the new name and tap **OK**. The file or directory is renamed on the filesystem immediately.

19.4.3 Creating a New Folder

Tap **New Folder**. An input dialog opens. Enter the folder name and tap **OK**. The new folder is created in the current directory.

19.4.4 Copy, Cut, and Paste

1. Select one or more files or folders.
2. Tap **Copy** to mark them for copying, or **Cut** to mark them for moving.
3. Navigate to the destination directory.
4. Tap **Paste**.

FairWindSK performs safety checks before paste operations:

- Copying a directory into itself or one of its subdirectories is blocked.
- Pasting into the source directory of a cut operation shows a warning.
- The self-paste safety check runs a self-test at startup to verify the detection logic is working correctly.

Directory copy and move operations are recursive — the entire subtree is copied or moved.

19.4.5 Deleting Files and Folders

Select one or more items and tap **Delete**. A confirmation dialog opens in the Bottom Bar horizontal drawer. Confirm to delete.

> File deletion is permanent. There is no recycle bin or undo for file operations. Take particular care when deleting directories — all contents are deleted recursively.

19.5 File List Columns

The file list shows three columns:

Column	Description
Name	Filename or directory name, with an icon indicating the file type (folder, image, document, etc.).

Size	File size in human-readable units (B, KB, MB, GB). Directories show the size of their immediate contents.
Date	Last modification timestamp. The *Type* column (column index 2) is hidden by default.

19.6 Practical Uses of the Files Tab

- Browse pilot book excerpts stored as PDFs on the Raspberry Pi.
- View passage planning images (harbour photos, aerial views, approach charts).
- Access voyage log files or NMEA recordings.
- Manage chart tile archives before registering them in the Charts tab.
- Review diagnostic log files generated by FairWindSK (see Chapter 29).

Part V

Settings

20 Settings — Overview

Open Settings by tapping **Settings** in the Bottom Bar. All configuration is arranged across ten tabs. Changes take effect immediately unless a restart is explicitly noted.

[SCREENSHOT]

Settings panel with all ten tabs visible in the tab bar

figures/settings-overview.png

Figure 20.1. Settings panel with all ten tabs visible in the tab bar

#	Tab name	Primary function	Restart needed?
0	Main	Window mode, language, UI scale, comfort preset, launcher grid, coordinate format	Language and virtual keyboard only
1	Top Bar	Drag-and-drop layout of the Top Bar instrument strip	No
2	Bottom Bar	Drag-and-drop layout of the Bottom Bar action strip	No
3	Widgets	Define and edit reusable data widget definitions	No
4	Comfort	Select and customise visual comfort presets	No
5	Connection	Signal K server URL, authentication token, mDNS discovery	No
6	Signal K	Map semantic names to actual Signal K paths	No
7	Units	Choose display units per measurement category	No

8	Applications	Manage the launcher: pages, tiles, folders, app details	No
9	System	Diagnostics, logging, performance, configuration management	No (some actions restart)

21 Settings — Main

21.1 Tab 0 — Main

Tab 0 — Main

The Main tab controls fundamental runtime behaviour: window display mode, language, UI element scale, active comfort preset, launcher grid dimensions, and coordinate format.

[SCREENSHOT]

Settings — Main tab: window mode, language, UI scale, comfort preset, launcher grid, coordinate format

`figures/settings-main.png`

Figure 21.1. Settings — Main tab: window mode, language, UI scale, comfort preset, launcher grid, coordinate format

21.2 Window Mode

Mode	Behaviour	Notes
Windowed	Resizable desktop window at the configured Left/-Top/Width/Height.	Development, testing, multi-monitor setups.
Centred	Centred window of the configured Width and Height. Position fields disabled.	Preferred windowed mode without an exact position.

Maximised	Full desktop work area.	On Raspberry Pi OS, uses the screen work area geometry directly, bypassing the window manager.
Full Screen	Frameless, always-on-top kiosk window.	Recommended for dedicated helm displays. On labwc desktops, set `autohide=true` in `wf-panel-pi.ini` if the panel overlays FairWindSK.

21.3 Language

FairWindSK ships with two fully supported interface languages. The language setting affects all native shell text — bar labels, settings pages, drawers, warnings, and status messages — as well as the `Accept-Language` header sent to embedded web content.

- **System default** — follows the OS locale when FairWindSK supports that language; falls back to English for every unsupported OS language.
- **English** — forces English regardless of OS locale.
- **Italiano** — forces Italian. All native shell text is displayed in Italian.

21.3.1 How localisation works

The `Localization` module normalises the selected language, chooses the effective `QLocale`, and builds the web `Accept-Language` header. The Qt translator (`.qm` resource) is loaded during startup, so native widgets and embedded web views start with the same language and culture. Translation source files live in `resources/i18n/` (e.g. `fairwindsk_it.ts` for Italian).

21.3.2 Adding or improving a language

See `docs/multilingual.md` in the repository for the full contributor workflow: adding translatable strings with `tr(…)`, updating `.ts` files with Qt Linguist, registering a new language in `Localization.cpp` and `ui/settings/Main.cpp`, and updating `CMakeLists.txt`.

> A restart is required after changing the language. FairWindSK shows an informational drawer when the language is changed. Without a restart, native Qt widgets and the embedded web view may use different languages.

21.4 UI Scale Preset

Four presets: Small, Normal (default), Large, Extra Large. When **Auto Scale** is ticked, FairWindSK selects the preset based on display resolution and DPI.

21.5 Comfort View Preset and Automatic Switching

Selects the active comfort preset directly from the Main tab. When **Auto Comfort View** is enabled, FairWindSK switches presets automatically using Signal K sun-position data (`environment.sun`). Both conditions must be met for auto mode to be available:

1. The `environment.sun` path must be configured in the Signal K tab.
2. The Signal K server must be connected and delivering values on that path.

21.6 Launcher Grid

Two spin boxes set the number of rows and columns of app tiles per launcher page. Fewer rows and columns produce larger tiles that are easier to tap. More produce more apps visible per page.

21.7 Coordinate Format

Three formats are available throughout the interface (Top Bar position widget, POB bar, Anchor bar, My Data waypoints):

Format	Example	When to use
DD.DDDDD°	48.85341° N	GPS logs, digital charting software.
DD°MM.MMM’	48°51.205’ N	Standard nautical format. Recommended for most sailors.
DD°MM’SS.S”	48°51’12.3” N	Traditional celestial navigation format.

21.8 Virtual Keyboard

Enables the Qt virtual keyboard for touchscreen-only deployments. A restart is required after toggling. On Raspberry Pi OS, the startup helper script applies the correct environment variable (`QT_IM_MODULE=qtvirtualkeyboard`) automatically.

22 Settings — Top Bar and Bottom Bar Editors

22.1 Tab 1 — Top Bar

Tab 1 — Top Bar

22.2 Tab 2 — Bottom Bar

Tab 2 — Bottom Bar

Tabs 1 and 2 share the same visual drag-and-drop editor — one for the Top Bar and one for the Bottom Bar.

[SCREENSHOT]

Settings — Top Bar editor: live preview strip, editing toolbar, and widget palette

`figures/settings-topbar-editor.png`

Figure 22.1. Settings — Top Bar editor: live preview strip, editing toolbar, and widget palette

22.3 Live Preview Strip

The preview at the top renders a full-width simulation of the bar using the current comfort preset colours and real widget data. Tap any item in the preview to select it.

22.4 Editing Toolbar Buttons

These are the exact icons defined in `BarLayoutSettings.cpp`.

Icon	Button	Action
	Minimize width	Widget uses only the space its content requires.
	Maximize width	Widget expands to fill all available space horizontally.
	Minimize height	Widget uses the standard bar height.
	Maximize height	Widget uses the full bar height (both header and value rows fully visible).
	Move left	Moves the selected widget one position to the left.
	Move right	Moves the selected widget one position to the right.
	Remove selected	Returns the selected widget to the palette.
	Reset defaults	Restores the bar to its built-in default layout.

22.5 Display Toggles

With a widget selected, four checkboxes control what is shown: **Show Icon**, **Show Text** (label), **Show Units**, **Show Trend** (arrow).

22.6 Widget Palette

The palette below the toolbar lists all available items. **Data widgets** are backed by Signal K paths. **Fixed shell controls** — Apps, POB, Autopilot, Anchor, Alarms, My Data, Settings, Signal K status, Clock — can be moved between the Top Bar and Bottom Bar. **Separators** ( ) and **Stretches** ( ) are invisible layout helpers.

23 Settings — Widgets

23.1 Tab 3 — Widgets

Tab 3 — Widgets

The Widgets tab manages the library of data widget definitions used by both bars.

[SCREENSHOT]

Settings — Widgets tab with widget list and definition fields

`figures/settings-widgets.png`

Figure 23.1. Settings — Widgets tab with widget list and definition fields

23.2 Widget Definition Fields

Field	Purpose
id	Unique identifier used internally and in bar layout entries.
name	Human-readable label shown in bars and this editor.
icon	Optional SVG icon path. Accepts Qt resource paths or filesystem paths.
type	Rendering format: `numerical`, `gauge`, `position`, `datetime`, or `waypoint`.
signalKPath	Full dot-separated Signal K data path.
sourceUnit	Unit in which Signal K delivers the value (e.g. `m/s`, `rad`).
defaultUnit	Unit to display in (e.g. `kn`, `deg`). Overridden by active unit preferences.

updatePolicy	`instant` (default) or `fixed`.
period	Target update period in milliseconds (default 1000).
minPeriod	Minimum update period in milliseconds (default 200).

24 Settings — Comfort

24.1 Tab 4 — Comfort

Tab 4 — Comfort

The Comfort tab is the visual theme editor for all six built-in comfort presets.

[SCREENSHOT]

Settings — Comfort tab: preset selector, colour swatches, and background image section

`figures/settings-comfort.png`

Figure 24.1. Settings — Comfort tab: preset selector, colour swatches, and background image section

24.2 The Six Comfort Presets

Icon	Preset	Conditions	Key characteristics
	Default	General fallback	Balanced mid-contrast. Usable in most conditions.
	Dawn	Around sunrise	Warm tones, slightly reduced brightness. Good contrast.
	Day	Full daylight, direct sun	Maximum brightness and contrast. For open-ocean sailing.
	Sunset	Around sunset	Warmer, transitional brightness.
	Dusk	Twilight	Reduced brightness, begins night-vision preservation.

	Night	Full dark, offshore watches	Minimum brightness, dark backgrounds, red accents. Preserves night vision. Alarms remain high-contrast.

24.3 Action Buttons

Icon	Button	Action
	Reset	Clears all custom colour overrides, background images, and QSS for the selected preset.
	Reset All	Clears customisation for all six presets.
	Export QSS	Saves the selected preset's effective stylesheet to a `.qss` file.
	Import QSS	Loads a `.qss` file as the custom stylesheet for the selected preset.

24.4 Colour Override Properties

Ten colour properties are customisable per preset:

Property	What it controls
Window background	Background colour of the Application Area and settings pages.
Text	Primary text colour throughout the interface.
Button background	Background of buttons and tool buttons in bars and drawers.
Button text	Text and icon tint on buttons.
Border	Borders, dividers, and frame outlines.
Accent	Primary accent colour for highlights and selected states.
Accent text	Text colour on accent-coloured backgrounds.

Icon	Default tint for monochrome SVG icons. Set to red for Night preset to aid night vision.
Scroll track	Background of scroll bar tracks.
Scroll knob	Colour of the scroll bar handle.

Never configure a preset that makes alarm text, POB information, or safety-critical readouts hard to read. Readability under all conditions takes precedence over aesthetics.

25 Settings — Connection

25.1 Tab 5 — Connection

Tab 5 — Connection

[SCREENSHOT]

Settings — Connection tab: server URL field, status line, action buttons, and console log

figures/settings-connection.png

Figure 25.1. Settings — Connection tab: server URL field, status line, action buttons, and console log

25.2 Server URL Field

An editable combo box pre-populated with the configured URL, `http://localhost:3000`, and `https://demo.signalk.org`. FairWindSK normalises the URL: prepends `http://` if no scheme is present, strips trailing slashes, and validates that only `http://` or `https://` are accepted. Typed URLs are committed to the saved configuration only when you tap **Connect**.

25.3 Status Line

A single line below the URL field shows the current token and connection state using three dot-separated fields: **State**, **Permission**, and **Expires**.

25.4 Action Buttons

25.4.1 Connect and Pause

Toggles the Signal K connection. **Connect** starts the connection; **Pause** suspends it without clearing the URL.

25.4.2 Request Token

Initiates the Signal K read/write authentication flow:

1. FairWindSK posts a new access request to `/signalk/v1/access/requests`.
2. On HTTP 202 + `PENDING`, FairWindSK opens the Signal K admin interface in the Bottom Bar drawer, navigated to the Security > Access Requests page.
3. FairWindSK polls the `href` every second until resolved.
4. On `COMPLETED` with `APPROVED`, the token is stored in `fairwindsk.ini` and injected as the `JAUTHENTICATION` cookie in the embedded browser's cookie store.

25.4.3 Cancel

Aborts the in-flight token request and clears the pending href.

25.4.4 Remove Token

Removes the stored token from memory, `fairwindsk.ini`, and the browser cookie store. Triggers a reconnect in public (unauthenticated) mode.

25.5 Console Log

A read-only scrollable text area below the buttons shows connection events: mDNS discovery messages, connection lifecycle events, and token request outcomes.

25.6 mDNS Automatic Discovery

On desktop platforms, FairWindSK scans the local network for Signal K servers using mDNS/Zeroconf on four service types: `_signalk-http._tcp`, `_signalk-https._tcp`, `_http._tcp`, and `_https._tcp`. Discovered servers are added to the URL combo box automatically. mDNS is disabled on Android and iOS builds.

26 Settings — Signal K Paths

26.1 Tab 6 — Signal K Paths

Tab 6 — Signal K Paths

Maps the semantic names that FairWindSK uses internally to the actual Signal K data paths on your server. Edit any field to change where FairWindSK reads or writes that value. Changes take effect after a short debounce delay with no restart required.

[SCREENSHOT]

Settings — Signal K Paths tab with label and editable path fields

`figures/settings-signalk.png`

Figure 26.1. Settings — Signal K Paths tab with label and editable path fields

26.2 Key Path Mappings

Key	Default Signal K path	Feature
`pos`	`navigation.position`	Top Bar position, POB position
`sog`	`navigation.speedOverGround`	Top Bar SOG widget
`cog`	`navigation.courseOverGroundTrue`	Top Bar COG widget
`hdg`	`navigation.headingTrue`	Top Bar HDG widget
`dpt`	`environment.depth.belowTransducer`	Top Bar depth widget

notifications.pob	notifications.mob	POB alarm
anchor.position	navigation.anchor.position	Anchor Watch
anchor.radius	navigation.anchor.currentRadius	Anchor Watch
anchor.max	navigation.anchor.maxRadius	Anchor Watch
anchor.rode	winches.windlass.rode	Anchor rode display
anchor.actions.dr	plugins.anchoralarm.dropAnchor	Anchor Drop button
anchor.actions.ra	plugins.anchoralarm.raiseAnchor	Anchor Raise button
anchor.actions.ra	plugins.anchoralarm.setRadius	Anchor Set Radius button
anchor.actions.up	plugins.windlassctl.up	Anchor Up button
anchor.actions.do	plugins.windlassctl.down	Anchor Down button
environment.sun	environment.sun	Auto comfort preset switching
autopilot.state	steering.autopilot.state	Autopilot bar mode display
autopilot.mode	steering.autopilot.mode	Autopilot engage commands
rsa	steering.rudderAngle	Autopilot rudder indicator

27 Settings — Units

27.1 Tab 7 — Units

Tab 7 — Units

[SCREENSHOT]

Settings — Units tab: server preset name and per-category override rows

figures/settings-units.png

Figure 27.1. Settings — Units tab: server preset name and per-category override rows

27.2 How Unit Preferences Work

FairWindSK reads unit preferences from the connected Signal K server. Each category row shows: the category label, an override combo box (your local preference), and the server's current value (read-only). If you choose the same unit as the server, no override is stored. If different, it is saved in `unitPreferences.overrides.CATEGOR` in `fairwindsk.json` and takes precedence.

28 Settings — Applications

28.1 Tab 8 — Applications

Tab 8 — Applications

A two-pane editor: the left pane manages launcher pages and folders; the right pane is the application palette.

[SCREENSHOT]

Settings — Applications tab: page tree (left) and app palette (right)

`figures/settings-apps.png`

Figure 28.1. Settings — Applications tab: page tree (left) and app palette (right)

28.2 Page Tree Actions

Icon	Action	What it does
	Add page	Creates a new empty launcher page.
	Page details	Opens the page details drawer (name, icon).
	Assign all apps	Adds all palette apps to the current page.
	Clear page	Removes all tiles from the current page.
	Clear all pages	Removes all tiles from every page.

28.3 Palette Actions

Icon	Action	What it does
	Create app	Opens the App Details drawer to create a new custom application.
	App details	Opens the App Details drawer to edit the selected application.
	Remove app	Permanently removes the selected application from the palette.
	Add to page	Adds the selected palette app to the current launcher page.

28.4 App Details Fields

Field	Description
Display name	Name shown on the launcher tile and in the Top Bar when the app is active.
URL	Full URL (`http://` or `https://`).
Icon path	Qt resource path, filesystem path, or bundled icon path.
Web zoom level	Percentage (default 100%). Increase for larger buttons; decrease for more content.
Active	Whether the tile is visible on the launcher.
Order	Secondary sort hint within a page.

29 Settings — System

29.1 Tab 9 — System

Tab 9 — System

The System tab is divided into two panes: **Performance** (left) and **Diagnostics and Logging** (right).

[SCREENSHOT]

Settings — System tab Performance pane: memory bars, per-core CPU, network, Raspberry Pi metrics

figures/settings-system-perf.png

Figure 29.1. Settings — System tab Performance pane: memory bars, per-core CPU, network, Raspberry Pi metrics

29.2 Performance Pane

29.2.1 Memory Readouts

- **Process memory** — resident RAM used by FairWindSK (from `/proc/self/statm` on Linux; Mach task API on macOS).
- **Total memory** — installed RAM (from `/proc/meminfo` on Linux; `sysctl hw.memsize` on macOS).
- **Memory usage bar** — process memory as a percentage of total RAM.

29.2.2 CPU Load

A row of horizontal progress bars, one per CPU core, updated every second. On Linux, data is read from `/proc/stat`; on macOS from the Mach `host_processor_info` API.

29.2.3 Network Addresses

Lists IPv4 addresses of all active, non-loopback network interfaces, refreshed every 10 seconds. Useful for confirming which IP address other devices should use to reach this installation.

29.2.4 Raspberry Pi Diagnostics

When the connected Signal K server exposes Raspberry Pi metrics (via the `signalk-server-raspberryp` plugin on path `environment.rpi`), a Raspberry Pi group box appears showing:

Metric	Description
CPU temperature	SoC CPU temperature in °C. Above 80°C indicates thermal throttling.
GPU temperature	VideoCore GPU temperature in °C.
CPU utilisation	Overall CPU load as a percentage.
Memory utilisation	RAM usage as a percentage.
SD utilisation	Storage usage on the SD card as a percentage.

29.3 Diagnostics and Logging Pane

[SCREENSHOT]

Settings — System tab Diagnostics pane: log level, persistent logs, interaction history, paths, View Logs and View Reports buttons

`figures/settings-system-log.png`

Figure 29.2. Settings — System tab Diagnostics pane: log level, persistent logs, interaction history, paths, View Logs and View Reports buttons

29.3.1 Log Level

Level	Messages included	When to use
Off (default)	No messages.	Normal operation. Minimises SD card writes on Raspberry Pi.

Critical	Fatal errors only.	Minimal logging with crash detection.
Warning	Critical + non-fatal anomalies.	Initial installation testing.
Info	Warning + lifecycle events.	Commissioning and setup.
Debug	Info + detailed internal state.	Troubleshooting. Significant overhead.
Full	All of the above + verbose tracing.	Developer use only. Very high overhead.

29.3.2 Persistent Logs

When enabled, FairWindSK writes a complete message log for each run to the per-user application data directory, with the process start timestamp in the filename.

29.3.3 Interaction History

When enabled, FairWindSK writes a lightweight journal of operator navigation and control actions alongside each run log. Useful for post-incident review.

29.3.4 View Logs and View Reports Buttons

Open the in-application log explorer in the Bottom Bar horizontal drawer. Log files appear on the left; parsed log entries appear on the right. Crash-report bundles (created automatically when FairWindSK did not exit cleanly) are accessible via **View Reports**.

29.4 Configuration Management Buttons

Button	Action
Reset	Discards unsaved changes and reloads the active configuration. FairWindSK keeps running.
Restore Defaults	Replaces all settings with the bundled defaults. All customisation is lost. Confirmation required.
Import	Imports a `fairwindsk.json` file from the filesystem. Validates, atomically replaces the active configuration, and reloads immediately.

Export	Copies the current `fairwindsk.json` to a chosen location.

Restore Defaults is irreversible without a backup. Export your configuration first if you may want to restore it.

29.5 System Control Buttons

Button	Action
Restart FairWindSK	Saves settings, then closes FairWindSK with exit code 1. The Raspberry Pi OS startup helper relaunches on exit code 1.
Quit FairWindSK	Saves settings and closes cleanly (exit code 0). The startup helper does **not** relaunch after a clean exit.

Quitting FairWindSK on a dedicated helm display leaves the display blank. Ensure another crew member is maintaining a safe watch before quitting.

Part VI

Reference

30 Comfort Presets and Visibility

Icon	Preset	Conditions	Key characteristics
	Default	General fallback	Balanced mid-contrast; usable in most conditions.
	Dawn	Around sunrise	Warm tones, slightly reduced brightness.
	Day	Full daylight, direct sun	Maximum brightness and contrast. For open-ocean sailing.
	Sunset	Around sunset	Warmer, transitional brightness.
	Dusk	Twilight	Reduced brightness; begins night-vision preservation.
	Night	Full dark, offshore watches	Minimum brightness, dark backgrounds, red-tinted icons. Preserves night vision. Alarms remain high-contrast on every preset.

30.1 Practical Guidance

- Use brighter, higher-contrast presets in daylight.
- Use dimmer, night-oriented presets when preserving night vision matters.
- Keep alarms and safety-critical text easy to read in every preset.
- Prefer readability over appearance. If a preset looks attractive but makes urgent information harder to see, switch to a clearer one.
- The Night preset sets the icon tint to red by default. This is intentional — red light minimises the time needed to re-adapt to darkness after looking at the display.

31 Display, Touch, and Rough-Conditions Operation

31.1 Display Readability

FairWindSK should remain readable in bright daylight, dusk, night, and rough conditions. If the display is difficult to read, correct that first before diagnosing any app issues.

Start with:

1. Screen brightness.
2. Comfort preset (correct for the current light conditions).
3. Viewing angle and helm position.
4. The app or page shown in the Application Area.

Keep safety-critical numbers easy to identify at a glance. Avoid presets that reduce contrast too much. Use larger displays for primary navigation whenever possible. Use a display sunshade or hood in direct sunlight.

31.2 Touch Use in Rough Conditions

- Brace yourself first. One hand on the boat, one on the screen.
- Use deliberate, single taps. Hesitant pressure and repeated rapid taps cause unintended actions.
- Never explore the autopilot controls by tapping speculatively — every command has an immediate physical effect.
- Keep the screen clean. Salt spray reduces touch accuracy significantly.
- Pre-configure what you need while conditions are calm.
- Prefer the large, stable targets of the Bottom Bar main icons.
- Confirm dialogs and mode changes before acting on them.

For helm use, stable layout and good readability matter more than decorative styling.

32 Recommended Operator Workflow

For most passages, this sequence keeps the shell predictable and easy to trust.

32.1 Before Departure

1. Start FairWindSK and confirm the shell health state is normal.
2. Verify that expected key data is live in the Top Bar.
3. Open the app most relevant to the current passage phase.
4. Confirm brightness and comfort preset.
5. Complete the pre-departure checklist (Chapter 2).

32.2 During Passage

1. Watch for stale or missing values in the Top Bar.
2. Cross-check route, depth, and heading whenever conditions change.
3. Use the launcher to switch tasks rather than forcing a stuck app.
4. Treat degraded states as operational warnings.
5. If the current screen stops feeling clear or trustworthy, tap **Apps** to return to the launcher.

32.3 After a Reconnect or Server Restart

1. Wait briefly for automatic recovery to complete.
2. Confirm the status shows *Live*.
3. Reopen the app if the foreground application was degraded.
4. Reconfirm key navigation values before depending on them.

32.4 Task Switching

Use the Bottom Bar rather than forcing the current app to do everything:

- Use Apps to return to the launcher and switch tasks.
- Use the open-apps strip to jump directly to a loaded application.
- Use operational bars (POB, Anchor, Autopilot) for vessel-operation tasks without leaving your navigation app.

33 Signal K Server on Docker

33.1 Official Docker Image

```
cr.signalk.io/signalk/signalk-server:latest
```

Supported architectures: `linux/amd64`, `linux/arm/v7`, `linux/arm64` (all Raspberry Pi 4 and 5 configurations).

33.2 Quick Start with Demo NMEA Data

```
mkdir $HOME/.signalk
docker run --init -it --rm \
  --name signalk-server \
  --publish 3000:3000 \
  -v $HOME/.signalk:/home/node/.signalk \
  cr.signalk.io/signalk/signalk-server \
  --sample-nmea0183-data
```

33.3 Production Deployment

```
docker run -d --init \
  --name signalk-server \
  -p 3000:3000 \
  -v $HOME/.signalk:/home/node/.signalk \
  cr.signalk.io/signalk/signalk-server
```

33.4 Updating Signal K

```
docker pull cr.signalk.io/signalk/signalk-server:latest
docker stop signalk-server
docker rm signalk-server
# Re-run the production docker run command above
```

34 Troubleshooting

34.1 I See Missing Values

Possible causes: the sensor or source instrument is offline; Signal K is not receiving that path; the value is not configured for display; the connection has dropped.

1. Check shell health state.
2. See whether other values are still live.
3. Confirm the source instrument is operating.
4. Check *Settings > Signal K Paths* — is the correct path configured?
5. Return to the launcher and reopen the affected app if needed.

34.2 I See Stale Values

Possible causes: temporary data interruption; reconnect in progress; network delay; partial degradation in the data source.

1. Treat the values cautiously.
2. Wait briefly for recovery.
3. Cross-check with other instruments.
4. Investigate if the stale state persists beyond 60 seconds.

34.3 A Web App Is Blank or Frozen

1. Retry once if the shell offers the option.
2. Return to the launcher.
3. Check shell health and Signal K connection state.
4. Use another app if the shell is otherwise healthy.

5. A failed application does not mean FairWindSK has failed.

34.4 The Shell Is Disconnected

1. Is Signal K running? Check LEDs on the Raspberry Pi; check the Docker container status: `docker ps -a | grep signalk`.
2. Is your device on the correct Wi-Fi network?
3. Is the server address in *Settings* > *Connection* correct?
4. Can a browser on another device reach `http://server-address:3000`?
5. Allow FairWindSK 60 seconds to attempt automatic reconnection.
6. Reconfirm all safety-critical values after reconnection.

34.5 No Applications Appear on the Launcher

1. Confirm status shows *Live*.
2. Are apps installed in the Signal K App Store?
3. In *Settings* > *Connection*, tap **Connect** to force a reconnect and app discovery.

34.6 Autopilot or Anchor Icon Is Greyed Out

1. Is the required plugin installed in the Signal K App Store?
2. Is the plugin enabled and configured in Signal K settings?
3. Is the correct plugin action path configured in *Settings* > *Signal K Paths*?

34.7 FairWindSK Running Slowly on Raspberry Pi

1. Check CPU temperature: `vcgencmd measure_temp`. Above 80°C indicates thermal throttling — improve cooling.
2. Check *Settings* > *System* > *Performance* for memory usage. FairWindSK should use under 250 MB at idle.
3. Use a Class A2 high-speed microSD card. A slow card is the most common cause of sluggish performance on Raspberry Pi.
4. Set Log Level to *Off* in *Settings* > *System* > *Diagnostics* to minimise SD card writes.

35 Daily Operating Checklist

Before Departure

☐	Check
☐	FairWindSK started — status shows *Live*
☐	Position live and plausible
☐	SOG updating — valid GPS fix
☐	Depth updating (if fitted)
☐	Heading showing
☐	Chart plotter open; vessel in correct position
☐	Route or waypoint active (if proceeding to a destination)
☐	AIS targets visible (if AIS fitted)
☐	Comfort preset set for current light conditions
☐	Screen brightness readable from helm
☐	POB button location confirmed
☐	Autopilot available and responding (if required)
☐	Anchor watch available (if anchoring tonight)

After Arrival

☐	Check
☐	Return to launcher — no active alarms
☐	Anchor watch set with appropriate radius (if anchoring)
☐	Comfort preset changed to Dusk or Night

☐	Track exported for logbook (if required)

36 Quick Reference Card

Immediate Action Guide

Situation	Immediate action	Then
Person Over Board	Tap **POB**. Once.	Call MAYDAY on VHF Ch 16. Deploy SAR route.
Alarm flashing	Tap **Alarms**. Read it.	Act per emergency procedures.
Screen frozen	Wait 10 s. Tap **Apps**.	Check status. Try reopening the application.
Shell disconnected	Wait 60 s for auto-reconnect.	If still disconnected: check Signal K and Wi-Fi.
Stale depth in shoal water	Cross-check immediately with a backup instrument.	Slow down. Use a hand lead or backup echo sounder.
Autopilot not responding	Tap **Stand By**. Take manual helm.	Investigate the cause before re-engaging.
Data stale everywhere	Check connection state.	Allow recovery or check Signal K server.
Data missing for one value	Cross-check with another source.	Check the sensor and the Signal K path.

One-Tap Navigation

Goal	Action
Return to home screen	Tap **Apps** — always available

Switch to another open app	Tap its icon in the open-apps strip
Change comfort preset	*Settings* > *Comfort*
Check connection status	Status area in Top Bar (right side)
Mark anchor position	Tap **Anchor** → **Drop**
Cancel anchor watch	Tap **Anchor** → **Raise**
Cancel a POB event	POB bar → select incident → **Cancel**
Export voyage track	*My Data* > *Tracks* → **Export**

37 Safety Notes

FairWindSK is an aid to navigation. It does not replace seamanship, lookout, paper or independently powered charts, or any certified safety procedure. The master and crew of the vessel remain solely responsible for the safe conduct of the voyage.

Always cross-check safety-critical information — position, depth, heading, collision risk — with independent instruments before acting on it. If FairWindSK contradicts what you see outside the boat or on trusted instruments, trust your eyes and independent navigation references first.

Missing depth is not zero depth. A missing value means the measurement is unavailable. Treat it as entirely unknown and navigate with full caution.

After a reconnect or server restart, reconfirm all key navigation values before depending on them. Give GPS, depth, wind, and heading sensors time to re-acquire and settle.

Autopilot commands must be confirmed against the vessel's actual behaviour. Never assume a mode change took effect because the button responded. Stand by to disengage and take manual helm at any moment.

Activating an alarm in FairWindSK raises a Signal K notification on the onboard network. It does not contact coast guard, MRCC, or any external authority. Make a DSC and/or voice distress call on VHF Channel 16 in any real emergency.

In an emergency, prioritise boat handling, crew safety, and direct observation over any interaction with an electronic display. A screen is a tool; your seamanship is your safety.

Safe sailing.

The OpenFairWind Team • University of Naples ’Parthenope’

https://github.com/OpenFairWind/FairWindSK

A Installation

This appendix covers every supported deployment scenario for FairWindSK, from a first-time source build on a developer laptop to a fully automatic kiosk boot on a dedicated helm display. Each scenario lists hardware or platform requirements, exact commands, and post-install verification steps.

> All scenarios assume a running Signal K server somewhere on the boat's Wi-Fi network. FairWindSK is a display shell, not a data server. Install and configure Signal K before installing FairWindSK.

A.1 Prerequisites

A.1.1 Common prerequisites for all desktop platforms

- CMake 3.16 or newer
- A C++17-capable compiler supported by the chosen Qt 6 kit
- Git
- Internet access during the first build (CMake downloads `nlohmann/json`, `QtZeroConf`, and `QHotkey` automatically; subsequent builds use the cached copies)

A.1.2 Required Qt 6 modules — desktop

Module	Purpose
`Core`	Qt base classes and event loop.
`Gui`	2D graphics and font handling.
`Widgets`	Widget toolkit used throughout the shell.
`Concurrent`	Thread pool for background tasks.

`Network`	HTTP/REST client and TLS.
`WebSockets`	Signal K WebSocket stream.
`Xml`	Configuration and resource parsing.
`Svg`	OpenBridge icon rendering.
`Positioning`	GPS location APIs (desktop fallback).
`WebEngineWidgets`	Embedded browser for hosted Signal K web apps.
`VirtualKeyboard`	On-screen keyboard for touchscreen deployments.
`PrintSupport`	PDF generation (crash reports).

A.1.3 Required Qt 6 modules — Android and iOS

Mobile builds replace `WebEngineWidgets` with a lighter web layer and drop desktop-only dependencies:

Module	Replaces or adds
`Qml`	QML runtime for WebView hosting.
`Quick`	Qt Quick scene graph.
`QuickWidgets`	`QQuickWidget` bridge between widget and QML worlds.
`WebView`	Replaces `WebEngineWidgets`. Uses the platform's native browser engine.

Mobile builds do *not* include `QtZeroConf`, `QHotkey`, `PrintSupport`, or `WebEngineWidgets`.

A.2 Generic Desktop Build (all platforms)

This is the baseline procedure that applies to every desktop platform before platform-specific steps are added.

1. Clone the repository:

```
git clone https://github.com/OpenFairWind/FairWindSK.git
cd FairWindSK
```

2. Configure:

```
cmake -S . -B build
```

3. Build (parallel across all CPU cores):

```
cmake --build build --parallel
```

4. Install to the default prefix:

```
cmake --install build
```

5. Override the install prefix if needed:

```
cmake -S . -B build -DCMAKE_INSTALL_PREFIX=/desired/prefix
cmake --build build --parallel
cmake --install build
```

6. If Qt is installed outside the default search path, pass `-DCMAKE_PREFIX_PATH` during configure:

```
cmake -S . -B build -DCMAKE_PREFIX_PATH=/path/to/Qt/6.x.x/gcc_64
```

> If internet access is restricted during the build, install `nlohmann_json` from your package manager first. CMake will use the system header instead of downloading it. On Debian/Ubuntu: `sudo apt install nlohmann-json3-dev`. On macOS with Homebrew: `brew install nlohmann-json`.

A.3 macOS Installation

A.3.1 Requirements

- macOS 12 Monterey or newer (recommended)
- Xcode command line tools: `xcode-select --install`
- Qt 6 desktop kit (via Qt Online Installer or Homebrew)
- CMake and Ninja (via Homebrew or Qt Maintenance Tool)

A.3.2 Install with Homebrew

```
brew install qt cmake ninja nlohmann-json
cmake -S . -B build -G Ninja \
  -DCMAKE_PREFIX_PATH="$(brew --prefix qt)"
```

```
cmake --build build --parallel
```

A.3.3 Run from the build tree

```
open build/FairWindSK.app
```

A.3.4 Install to the system

```
cmake --install build
# Installs to /usr/local by default:
open /usr/local/FairWindSK.app
```

A.3.5 Redistribute (deploy Qt runtime)

For distribution outside the build tree, use Apple's deployment tool:

```
macdeployqt build/FairWindSK.app
```

This bundles all Qt frameworks inside the `.app` so the app runs on machines without Qt installed.

A.3.6 Post-install verification

1. Launch FairWindSK.
2. Open *Settings* > *Connection*. Enter your Signal K server address and tap **Connect**.
3. Confirm the status changes to *Live* and apps appear on the launcher.

A.4 Linux Desktop Installation

A.4.1 Requirements (Debian / Ubuntu)

```
sudo apt update
sudo apt install \
  build-essential cmake ninja-build git \
  nlohmann-json3-dev \
  qt6-base-dev qt6-base-dev-tools \
  qt6-webengine-dev qt6-webengine-dev-tools \
  qt6-websockets-dev qt6-positioning-dev \
  libqt6svg6-dev qt6-virtualkeyboard-dev \
  qt6-base-private-dev libqt6printsupport6 \
  libavahi-compat-libdnssd-dev libnss-mdns avahi-utils
```

A.4.2 Build and run

```
cmake -S . -B build -G Ninja
cmake --build build --parallel
./build/FairWindSK
```

A.4.3 System install

```
sudo cmake --install build
```

`cmake --install` on a Linux desktop target:

- Installs the `FairWindSK` binary to `${CMAKE_INSTALL_BINDIR}` (default `/usr/local/bin`).
- Installs the bundled icon directory alongside the binary.
- Installs the desktop-only helper libraries to `${CMAKE_INSTALL_LIBDIR}`.
- Installs the `fairwindsk.desktop` launcher file to the standard XDG applications directory.
- Installs the `fairwindsk.png` menu icon to `/usr/local/share/icons/hicolor/256x256/apps/`.

After installation, FairWindSK appears in the applications menu. Run it from the menu or with:

```
FairWindSK
```

A.4.4 Packaging

Include the Qt runtime and the external dependencies built under `build/external/lib` in your package. The build-tree rpath means the binary runs from `build/` without `LD_LIBRARY_PATH`, but a packaged install needs those libraries in the system library path or bundled beside the binary.

A.5 Raspberry Pi OS — Navigation Station

This scenario installs FairWindSK on a Raspberry Pi used as a normal desktop navigation station — screen, keyboard, and mouse attached, running the Raspberry Pi OS desktop.

A.5.1 Requirements

Raspberry Pi 4 or 5 with Raspberry Pi OS Bookworm (64-bit recommended), and a stable internet connection for the initial package download.

A.5.2 Install build dependencies

```
sudo apt update
sudo apt install \
  build-essential cmake ninja-build git \
  nlohmann-json3-dev \
  qml6-module-qt-labs-folderlistmodel \
  qml6-module-qtquick-window \
  qml6-module-qtquick-layouts \
  qml6-module-qtqml-workerscript \
  libnss-mdns avahi-utils \
  libavahi-compat-libdnssd-dev libxkbcommon-dev \
```

```
qt6-base-dev qt6-base-dev-tools \
qt6-webengine-dev qt6-webengine-dev-tools \
qt6-websockets-dev qt6-positioning-dev \
libqt6svg6-dev \
qt6-virtualkeyboard-dev \
libqt6virtualkeyboard6 \
qt6-virtualkeyboard-plugin
```

If the build fails with a missing `Qt6VirtualKeyboardConfigVersionImpl.cmake`, apply the known workaround:

```
sudo touch /usr/lib/aarch64-linux-gnu/cmake/Qt6VirtualKeyboard/\
Qt6VirtualKeyboardConfigVersionImpl.cmake
```

Then re-run `cmake`.

A.5.3 Build and install

```
git clone https://github.com/OpenFairWind/FairWindSK.git
cd FairWindSK
cmake -S . -B build -G Ninja
cmake --build build --parallel
sudo cmake --install build
```

FairWindSK appears in the application menu under *Navigation* or *Internet*.

A.5.4 OpenPlotter integration

If OpenPlotter is detected during `cmake --install`, FairWindSK also copies its launcher icon into the OpenPlotter menu directories. No extra steps are required.

A.6 Raspberry Pi OS — Kiosk Helm Display

This is the recommended scenario for a dedicated helm display: a Raspberry Pi 4 or 5 with a touchscreen, running FairWindSK automatically at boot with no keyboard and no desktop menu visible.

A.6.1 Overview

1. Install FairWindSK as in Section A.5.
2. Configure FairWindSK for full-screen kiosk mode.
3. Install the `fairwindsk-startup` helper to manage autostart.
4. (Optional) Configure the desktop panel to auto-hide.
5. Reboot and verify.

A.6.2 Step 1 — Build and install

Follow Section A.5. On Raspberry Pi OS, `cmake --install` automatically installs the `fairwindsk-startup.desktop` autostart entry to `/etc/xdg/autostart/`. FairWindSK will start automatically on the next login.

A.6.3 Step 2 — Configure full-screen kiosk mode

Open *Settings* > *Main* and set **Window Mode** to **Full Screen**. This requests a frameless, always-on-top window that covers the entire screen with no title bar or window decorations.

Alternatively, edit `fairwindsk.json` directly:

```
{
  "main": {
    "windowMode": "fullscreen"
  }
}
```

A.6.4 Step 3 — Install the startup helper

The `fairwindsk-startup` Python script manages the FairWindSK process lifecycle. It is installed by `cmake --install` alongside the binary.

What `fairwindsk-startup` does at each boot:

1. Reads `fairwindsk.ini` to find the configuration path.
2. Reads `fairwindsk.json` to check the `virtualKeyboard` flag and sets `QT_IM_MODULE=qtvirtualkeyboa` accordingly.
3. Waits up to 45 seconds for the Signal K web-app catalogue endpoint to respond (`/signalk/v1/apps/list` or the legacy `/skServer/webapps`). This ensures plugin icons and artwork are available before FairWindSK starts.
4. Launches FairWindSK.
5. If FairWindSK exits with code 1 (the *Restart* button in *Settings* > *System*), the helper relaunches it immediately.
6. If FairWindSK exits with code 0 (the *Quit* button or a clean shutdown), the helper exits cleanly.

> Use **Restart FairWindSK** in *Settings* > *System* (exit code 1) to relaunch after configuration changes. **Quit FairWindSK** (exit code 0) leaves the display blank without a restart.

A.6.5 Step 4 — Hide the desktop panel (labwc / Bookworm)

On Raspberry Pi OS Bookworm with the labwc compositor, the taskbar panel can overlay the full-screen FairWindSK window. To prevent this, configure the panel to auto-hide:

```
# Edit the panel configuration file
nano ~/.config/wf-panel-pi.ini
```

Add or change:

```
[panel]
autohide = true
```

Save the file and reboot.

A.6.6 Step 5 — Enable virtual keyboard (touchscreen only)

If the helm display has no physical keyboard, enable the Qt virtual keyboard:

1. Open *Settings* > *Main*.
2. Tick **Virtual Keyboard**.
3. Tap **Restart FairWindSK** in *Settings* > *System*.

The startup helper reads the setting at each boot and sets the environment variable automatically.

A.6.7 Step 6 — Verify the kiosk boot

1. Reboot the Raspberry Pi.
2. Verify FairWindSK starts automatically (allow up to 60 seconds for Signal K and FairWindSK to start together).
3. Confirm the display is full-screen with no title bar.
4. Confirm the status shows *Live*.
5. Confirm apps appear on the launcher.

> On a Raspberry Pi 4 with a class A2 microSD card, FairWindSK typically becomes interactive in under 30 seconds from power-on, assuming Signal K starts quickly. Use a fast card (>90 MB/s read) to minimise boot time.

A.7 Windows Installation

A.7.1 Requirements

- Windows 10 or 11 (64-bit)
- Qt 6 desktop kit for MSVC (installed via the Qt Online Installer)
- Visual Studio Build Tools 2022 or Visual Studio Community 2022
- CMake and Ninja (bundled with Qt or installed separately)

A.7.2 Build and install

Open a Developer Command Prompt with the MSVC environment loaded, then:

```
git clone https://github.com/OpenFairWind/FairWindSK.git
cd FairWindSK
cmake -S . -B build -G Ninja ^
  -DCMAKE_PREFIX_PATH="C:\Qt\6.x.x\msvc2022_64"
cmake --build build --parallel
cmake --install build
```

A.7.3 Deploy the Qt runtime

After a successful build, deploy the Qt runtime DLLs beside the executable:

```
windeployqt build\FairWindSK.exe
```

`cmake --install` installs the executable, the bundled icon directory under `bin\icons`, and the desktop-only helper DLLs into the same `bin` directory.

A.7.4 Run

```
build\FairWindSK.exe
```

> `file://` launcher entries are retained in configuration for compatibility but are blocked by the single-window model on Windows. Use `http://` or `https://` URLs for all applications.

A.8 Android Installation

Android builds use the Qt WebView mobile implementation. Qt Creator is the recommended workflow for Android because it manages the SDK, NDK, APK signing, and device deployment.

A.8.1 Requirements

- A macOS, Linux, or Windows build host
- Qt 6 with the Android kit for your target ABI (arm64-v8a is recommended for modern devices)
- Android Studio (for the SDK, NDK, and emulator)
- Java JDK (bundled with Android Studio or installed separately)
- CMake and Ninja

A.8.2 Step-by-step build guide

1. Open Android Studio and install: an Android SDK platform, SDK Platform-Tools, SDK Build-Tools, and the NDK version recommended by your Qt re-

lease.

2. In the Qt Maintenance Tool, install the Android Qt components that match your host and target ABI.
3. In Qt Creator, go to *Preferences* > *Devices* > *Android* and verify that the SDK, NDK, JDK, and platform paths are all detected correctly.
4. Clone the repository:

```
git clone https://github.com/OpenFairWind/FairWindSK.git
```

5. Open `FairWindSK/CMakeLists.txt` in Qt Creator.
6. Select an Android kit (e.g. *Android Qt 6.x.x Clang arm64-v8a*).
7. Let Qt Creator configure the project. The mobile configuration uses `Qt::WebView`, `Qt::Quick`, and `Qt::QuickWidgets`, and skips desktop-only dependencies.
8. Build with *Build* > *Build Project*.
9. Connect an Android device with developer mode enabled, or start an Android emulator.
10. Deploy with *Build* > *Run*.

A.8.3 Command-line configure (advanced)

```
cmake -S . -B build-android -G Ninja \
  -DCMAKE_TOOLCHAIN_FILE="$ANDROID_NDK/build/cmake/android.toolchain.cmake" \
  -DANDROID_ABI=arm64-v8a \
  -DANDROID_PLATFORM=android-24 \
  -DCMAKE_PREFIX_PATH="/path/to/Qt/6.x.x/android_arm64_v8a"
cmake --build build-android --parallel
```

Qt Creator is still recommended for packaging and device deployment.

A.8.4 Android feature differences

Feature	Android status
Hosted web apps	Supported via Qt WebView.
Signal K stream	Supported (WebSockets).
mDNS server discovery	Not available (disabled on Android).
Global hotkey SHIFT+TAB	Not available (desktop-only, uses QHotkey).
Virtual keyboard	Uses Android system keyboard; Qt Virtual Keyboard setting has no effect.
File printing / PDF	Not available (PrintSupport not included).

QWebEngine cookies	Not used; mobile path uses a separate mechanism.
`file://` apps	Blocked by the single-window model on all targets.

A.9 iOS and iPadOS Installation

iOS and iPadOS builds use the same mobile web implementation as Android. Qt Creator with an Apple developer account is required for device deployment.

A.9.1 Requirements

- A macOS build host
- Xcode and Xcode command line tools
- Qt 6 with the iOS kit
- CMake and Ninja
- An Apple developer account and provisioning profile (for device deployment; not needed for the Simulator)

A.9.2 Step-by-step build guide

1. Open Xcode at least once and accept the license agreement.
2. In the Qt Maintenance Tool, install the iOS Qt components that match your desktop Qt release.
3. Clone the repository:

```
git clone https://github.com/OpenFairWind/FairWindSK.git
```

4. Open `FairWindSK/CMakeLists.txt` in Qt Creator.
5. Select an iOS kit (e.g. *Qt 6.x.x for iOS*).
6. Build with *Build > Build Project*.
7. To run on the Simulator: choose an iOS Simulator run target and select *Build > Run*.
8. To run on a device: connect the iPhone or iPad, select the physical device in the run target chooser, verify signing settings when prompted, and run from Qt Creator.

A.9.3 Command-line configure (advanced)

```
cmake -S . -B build-ios -G Ninja \
  -DCMAKE_SYSTEM_NAME=iOS \
```

```
  -DCMAKE_OSX_SYSROOT=iphoneos \
  -DCMAKE_OSX_ARCHITECTURES=arm64 \
  -DCMAKE_PREFIX_PATH="/path/to/Qt/6.x.x/ios"
cmake --build build-ios --parallel
```

Use `iphonesimulator` and `x86_64` (or `arm64` on Apple Silicon hosts) for Simulator builds.

A.9.4 iOS feature differences

Feature	iOS / iPadOS status
Hosted web apps	Supported via Qt WebView (WKWebView).
Signal K stream	Supported (WebSockets).
mDNS server discovery	Not available. Enter the server URL manually.
Global hotkey SHIFT+TAB	Not available.
Virtual keyboard	Uses iOS system keyboard.
File printing / PDF	Not available.
`file://` apps	Blocked by the single-window model on all targets.

A.10 Signal K Server on Docker

If you do not already have a Signal K server, Docker is the fastest way to get one running. This section documents the Docker deployment from the FairWindSK project repository.

A.10.1 Docker image

```
cr.signalk.io/signalk/signalk-server:latest
```

Supported architectures: `linux/amd64`, `linux/arm/v7`, `linux/arm64` (covers all Raspberry Pi 4 and 5 configurations).

A.10.2 Quick start with demo NMEA data

```
mkdir $HOME/.signalk
docker run --init -it --rm \
  --name signalk-server \
  --publish 3000:3000 \
  -v $HOME/.signalk:/home/node/.signalk \
  cr.signalk.io/signalk/signalk-server \
```

```
  --sample-nmea0183-data
```

This runs Signal K on port 3000 with simulated NMEA 0183 data. Use it to test FairWindSK without real instruments.

A.10.3 Production deployment

```
docker run -d --init \
  --name signalk-server \
  -p 3000:3000 \
  -v $HOME/.signalk:/home/node/.signalk \
  cr.signalk.io/signalk/signalk-server
```

- The `-d` flag runs the container as a background daemon.
- The `-v` flag mounts the configuration directory so Signal K settings, plugins, and data persist across container restarts.
- On first access, the Signal K admin web UI at `http://localhost:3000` prompts you to create admin credentials.

A.10.4 Updating Signal K

```
docker pull cr.signalk.io/signalk/signalk-server:latest
docker stop signalk-server
docker rm signalk-server
# Re-run the production docker run command above
```

A.10.5 Directory structure inside the container

Path	Contents
`/home/node/signalk`	Signal K server files (read-only inside container).
`/home/node/.signalk`	Settings files, plugins, and data. **Mount this to the host.**

A.11 First-Run Configuration

After installation on any platform, follow these steps on the first launch.

A.11.1 Configuration file locations

FairWindSK stores its configuration in two files in the per-user Qt configuration directory:

File	Contents
`fairwindsk.json`	All user-visible settings: connection URL, bar layouts, app catalogue, widget definitions, comfort presets, units, and Signal K path mappings. Human-readable JSON. Safe to back up, transfer, and version-control.
`fairwindsk.ini`	Qt-managed settings: path to `fairwindsk.json`, the debug flag, and the authentication token. The token is kept here intentionally so it is never included in a configuration export.

On first run, if `fairwindsk.json` does not exist, FairWindSK seeds it from the bundled defaults in `resources/json/configuration.json`.

A.11.2 Connect to Signal K

1. Open *Settings* > *Connection*.
2. Enter the Signal K server address, e.g. `http://192.168.1.100:3000`. If you are running Signal K on the same device, use `http://localhost:3000`.
3. Tap **Connect**.
4. Wait for the status to change to *Live*.
5. If the server requires authentication, tap **Request Token** and approve the request in the Signal K admin interface.

A.11.3 Authenticate (optional)

If the Signal K server has security enabled:

1. Tap **Request Token** in *Settings* > *Connection*.
2. FairWindSK opens the Signal K admin interface in the Bottom Bar drawer, navigated to *Security* > *Access Requests*.
3. Approve the request as an admin user.
4. FairWindSK stores the token automatically and reconnects.

A.11.4 Essential first-time settings

Setting	Where and what to set

Units	*Settings* > *Units*. Choose knots (speed), nautical miles (distance), and metres or feet (depth).
Coordinate format	*Settings* > *Main*. Choose DD°MM.MMM' for standard nautical use.
Window mode	*Settings* > *Main*. Choose Full Screen for a dedicated helm display; Windowed or Centred for a navigation station.
Comfort preset	*Settings* > *Comfort*. Set Day for initial testing; change to Night for the night watch.
Launcher apps	*Settings* > *Applications*. Arrange your chart plotter, instruments, and other apps on page 1.
Virtual keyboard	*Settings* > *Main* (touchscreen only). Tick, then restart FairWindSK.

A.12 Key Configuration Fields

Advanced users and system integrators can edit `fairwindsk.json` directly. This section covers the most useful fields.

A.12.1 Connection

```
{
  "connection": {
    "server": "http://your-signalk-host:3000"
  }
}
```

A.12.2 Window mode

```
{
  "main": {
    "windowMode": "fullscreen",
    "language": "en",
    "virtualKeyboard": false
  }
}
```

Valid values for `windowMode`: `windowed`, `centered`, `maximized`, `fullscreen`. Valid values for `language`: `system`, `en`, `it`.

A.12.3 Units

```
{
  "units": {
    "vesselSpeed": "kn",
    "windSpeed":   "kn",
```

```
    "distance":    "nm",
    "depth":       "mt"
  }
}
```

A.12.4 Signal K path overrides

```
{
  "signalk": {
    "sog": "navigation.speedOverGround",
    "dpt": "environment.depth.belowTransducer",
    "notifications.pob": "notifications.mob"
  }
}
```

A.12.5 Diagnostics

```
{
  "diagnostics": {
    "logLevel":          "off",
    "persistentLogs":    true,
    "interactionHistory": true,
    "email": "your-email@example.com"
  }
}
```

Valid log levels: `off`, `critical`, `warning`, `info`, `debug`, `full`.

A.13 Troubleshooting Installation Problems

A.13.1 FairWindSK does not start after installation

1. Run the binary from a terminal to see error output: `FairWindSK` (Linux/macOS) or `build\FairWindSK.exe` (Windows).
2. Check that Qt WebEngine libraries are available. On Linux, run `ldd build/FairWindSK | grep "not found"`.
3. On Raspberry Pi, check for the virtual keyboard CMake workaround (Section A.5).

A.13.2 No applications appear on the launcher

1. Verify the Signal K server address in *Settings* > *Connection*.
2. Confirm the server exposes `/signalk/v1/apps/list` or `/skServer/webapps`.
3. Check the Signal K App Store — at least one web app must be installed on the server for it to appear in FairWindSK.
4. Enable debug logging: add `debug=true` to `fairwindsk.ini` and relaunch. Logs include connection attempts and app discovery details.

A.13.3 Build fails with missing Qt module

1. Verify the Qt version: `qmake6 --version` or `cmake --find-package -DNAME=Qt6 -DCOMPILER_ID=GNU …`
2. Check that all required packages are installed (Sections A.4 and A.5).
3. If `CMAKE_PREFIX_PATH` is needed, pass it explicitly: `cmake -S . -B build -DCMAKE_PREFIX_PATH=/pa`

A.13.4 FairWindSK does not autostart on Raspberry Pi

1. Verify the autostart file was installed:

```
ls /etc/xdg/autostart/fairwindsk-startup.desktop
```

2. Verify the `fairwindsk-startup` script is executable and in PATH:

```
which fairwindsk-startup
```

3. Run the startup helper manually from a terminal to check for errors:

```
fairwindsk-startup
```

4. Ensure the desktop session supports XDG autostart (the default Raspberry Pi OS desktop does).